defumat

what it computes, and how to ask for it

Plane-wave DFT in Python and JAX, validated against Quantum ESPRESSO

September 12, 2026

Abstract

User documentation for the feature set: every capability, the equation behind it, a snippet that runs it, what it was validated against, and *what it refuses*. Two conventions run through the whole document.

Everything is validated against Quantum ESPRESSO on the same input, and the agreement is quoted per feature rather than claimed in general. Where a number is missing it is because the comparison does not exist, and that is said rather than hidden.

Anything not implemented is refused by name, with an error saying what is missing — never silently approximated. The "Refuses" notes are as much of the documentation as the features: they are the promise that a run which starts is a run whose physics is there.

One equation is written down and the rest are derived from it. Almost everything below is a derivative of the total energy, taken by the compiler rather than by hand:

$$E[\{\psi\},\tau,\varepsilon] = T_s + E_{\mathrm{H}} + E_{xc} + E_{\mathrm{loc}} + E_{\mathrm{nl}} + E_{\mathrm{Ewald}}$$

quantity	what it is	how it is obtained
$v_{xc}(\mathbf{r})$	$\delta E_{xc}/\delta n$	`jax.grad` of the energy density
$\mathbf{F}_I$	$-\partial E/\partial \boldsymbol{\tau}_I$	`jax.grad` at frozen ψ
σ_{ij}	$-\Omega^{-1}\partial E/\partial \varepsilon_{ij}$	`jax.grad` along a strain
$C_{I\alpha,J\beta}$	$\partial^2 E/\partial\tau_{I\alpha}\partial\tau_{J\beta}$	`jvp` of the force
$Z^*_{I,\alpha\beta}$	$\partial F_{I\alpha}/\partial\mathcal{E}_\beta$	`jvp` of the force along the field response
$\partial\epsilon_{ij}/\partial\tau$	Raman	`jvp` of the second-order energy

The "frozen ψ" is what makes this exact rather than approximate: at self-consistency the energy is stationary with respect to the wavefunctions, so the terms from $\partial\psi/\partial\lambda$ vanish. That is the Hellmann–Feynman argument, and taking it one order up is the $2n{+}1$ theorem, which is why a *third* derivative needs only first-order wavefunctions.

Contents

1 Input and invocation

defumat reads **`pw.x` input files**. The same file that runs in Quantum ESPRESSO runs here, which is what makes every comparison in this document possible.

```
from defumat import Calculator

calc = Calculator.from_file("scf.in", pseudo_dir="pseudo", conv_thr=1.0e-8)
result = calc.get_scf()

print(result.total_energy, "Ry in", result.iterations, "iterations")
```

`conv_thr` means what it means in a `pw.x` input: it is compared against QE's `dr2`, the Hartree energy of the density residual.

It need not be given at all. `from_file` and `from_text` **adopt the input's own `&electrons` namelist** — `conv_thr`, `mixing_beta`, `mixing_mode`, `mixing_fixed_ns` and `electron_maxstep` — so an input file that states how to converge itself converges the same way here as under `pw.x`. A keyword argument given to the constructor still wins over the file, which is how the snippet above overrides it. One variable of that namelist is read by `pw.x` and deliberately *not* adopted: `diagonalization`, because this package offers one eigensolver, so honouring `diagonalization = 'cg'` would turn a run that works into an error, and mapping it silently onto Davidson would be the kind of substitution refused everywhere else here. Name a solver at construction instead.

The adopted namelist reaches a relaxation too, and did not until 2026-09-11: `electron_maxstep`, `diago_full_acc` and `mixing_fixed_ns` were read off the file and then dropped on the way into `get_relax`, so every SCF inside a relaxation ran at the default hundred iterations whatever the input asked for. On a DFT+U relaxation the dropped `mixing_fixed_ns` decides which minimum of the $+U$ functional the run lands in, so the relaxed *geometry* could differ with nothing saying the request had been ignored.

An input variable that changes the physics and is not implemented is refused by name, never read and ignored — a run that quietly drops one is wrong by far more than the agreement with `pw.x` this code is checked to. That covers a charged cell (`tot_charge`), the saw-tooth field and the dipole correction (`tefield`, `dipfield`), the Berry-phase finite field (`lelfield`), the isolated-system corrections (`assume_isolated`), two chemical potentials (`twochem`, in either namelist), per-orbital atomic occupations (`one_atom_occupations`), the constant-cutoff kinetic functional (`qcutz`), a hand-pinned FFT grid (`nr1`–`nr3s`), and the four **symmetry** switches `no_t_rev`, `force_symmorphic`, `use_all_frac` and `nosym_evc`.
Those last four were read by *nothing* until 2026-09-11. Each says that some part of the group is not a symmetry of the state being asked for, so ignoring one left the k-mesh reduced with operations the input had excluded and `sym_rho` averaging over them — a total energy that will not match a benchmark generated from the same file, with nothing saying why. Anything QE defaults to is silent, so an ordinary input never reaches the refusal.

One object, bound methods

A `Calculator` is a system together with its pseudopotentials, and every calculation in this document is a method on it. The pseudopotentials are read from the names the `ATOMIC_SPECIES`

card gave, resolved against `pseudo_dir` — which `from_file` defaults to the input file's own directory — so a script names an input and then names physics:

```
calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")

bands = calc.get_bands(kpoints=path)      # runs the SCF first if none is cached
dos = calc.get_dos(grid=(8, 8, 8))
forces = calc.get_forces()
epsilon = calc.get_dielectric_tensor()
phonons = calc.get_phonons()

bands.plot()                              # and each result draws itself
```

Three properties are worth knowing before relying on it.

The ground state is cached, and computed on demand. A method that needs one and finds no cache runs an SCF, printing a line to stderr saying so; `get_scf()` is the explicit path and is the same code. The cache is a *single slot* holding the result together with the options that produced it — calling `get_scf` again with different options reruns and replaces it — because what it holds is the wavefunctions, and a cache keyed by option sets would quietly hold several sets of them. `calc.scf_result` reads the slot without triggering anything, and a state that did not converge is refused rather than differentiated.

Nothing mutates. `with_positions`, `with_cell` and `with_spin` return a *new* calculator with an empty cache; the converged state crosses as a starting guess where that is sound, which for `with_spin` is the spin-regime continuation described under *Spin, magnetism and spin-orbit* and converges in one iteration rather than twenty-five. A cached result under a moved atom would otherwise answer for the geometry it was converged at.

The refusals pass through untouched. Every restriction in this document — PAW Born charges, a metal's Raman tensor, a spiral with spin-orbit coupling — raises through the bound method exactly as through the function beneath it.

The functional entry points the rest of this document names — `run_scf`, `run_bands`, `dielectric_tensor` and the others — are unchanged and remain the way to drive a calculation whose state is being managed by hand:

```
from defumat.io.pwin import read_pw_input
from defumat.pseudo import read_upf
from defumat.scf.driver import Calculation, run_scf
from defumat.system import build_system

system = build_system(read_pw_input("scf.in"))
pseudos = tuple(read_upf(f"pseudo/{s.pseudo_file}") for s in system.structure.species)
result = run_scf(system, pseudos, conv_thr=1.0e-8)
```

What such a call has to carry with the density is the rest of the converged state — `becsum` for a PAW dataset, `ns` under a Hubbard U, `tau` under a meta-GGA. None of the three can be rebuilt from the density, and the entry points refuse without them rather than computing something else. The calculator passes them.

A command line covers the common workflows:

```
python3 -m defumat.cli inspect <qe-output>   # summarise what the parser reads
python3 -m defumat.cli dos     <input>       # density of states
```

```
python3 -m defumat.cli pdos    <input>       # projected DOS
python3 -m defumat.cli relax   <input>       # structural relaxation
python3 -m defumat.cli stress  <input>       # stress tensor
python3 -m defumat.cli spiral  <input>       # spin-spiral scan
```

Standard `pw.x` variables — `ecutwfc`, `ecutrho`, `ibrav`, `nbnd`, `occupations`, `degauss` and the rest — mean what they mean there and are not re-documented here. A handful of knobs are worth knowing because they are specific to this code or to a feature below:

`mbj_c`	fixes Tran–Blaha's c instead of taking the cell average
`london_s6`, `london_rcut`	Grimme D2's scaling and real-space cutoff
`cell_dofree`	which cell degrees of freedom a vc-relax may move
`starting_ns_eigenvalue`	seeds the DFT+U occupation matrix
`spiral_q`	the spin-spiral wavevector, in lattice coordinates
`LOCAL_MAGNETIC_FIELDS`	a card with no `pw.x` counterpart

Units are Rydberg atomic units throughout (Ry, bohr), matching QE. The only layer that speaks anything else is `io/`: Hubbard U in the `HUBBARD` card is in eV and is converted at the boundary, as `pw.x` does it.

What a run will cost, before it is run

`Calculation` builds the G-vectors, the plane waves and the projectors on the device, so an input too large for the machine dies in setup without saying what was too large. `Calculator.estimate()` answers that from the cutoffs and the datasets alone, allocating nothing:

```
print(Calculator.from_file("scf.in").estimate().report())
```

or `python3 -m defumat.cli size scf.in`. **It sizes the run the input describes**, which means it reads that input's own `diago_david_ndim` and resolves `k_batch` the way the SCF would — an estimate that quietly assumed a library default would report a run that does not happen, and nobody relying on one is in a position to notice. The arguments and the CLI flags are *overrides* of the file, and the report's header states which assumptions it used. It reports n_{gm}, n_{gms}, n_{pwx}, n_{bnd}, n_{kb} and both FFT grids — *exactly*, from the same predicates the setup selects with, which is asserted against a real `Calculation` rather than trusted — and a floor on the bytes for the wavefunctions, the projectors, the eigensolver's subspace and the fields. The G-vector enumeration runs in slabs, so an input destined for a GPU can be sized on a laptop.

Two lines of that report are worth reading before believing a hand estimate. The Davidson subspace is $2\,n_{vecx}\,n_{dim}$ for ψ and $H\psi$ at QE's $n_{vecx} = 4\,n_{bnd}$, plus the Ritz block, and it is usually the largest single array — on a 157-atom slab at $E_{\mathrm{cut}} = 60$ Ry it is 139 GB against 23 GB of wavefunctions. And the eigensolver is *not* doubled by spin, because the channels are solved one after another, where the wavefunctions are; the report separates the two.

The one lever that changes that floor without changing the physics is `diago_david_ndim` — QE's own name for it, read from `&electrons`, and settable per call or at construction:

```
calc = Calculator.from_file("scf.in", david=2)    # or diago_david_ndim = 2
```

It halves ψ and $H\psi$ exactly, for one more SCF iteration on silicon and the same total energy to 7.3e-11 Ry — the trade QE's documentation describes. Note that `INPUT_PW.txt` gives its default as 2 and is stale: `input_parameters.f90` sets 4 and `input.f90` assigns it straight through, so the number `pw.x` runs is **4**, and that is the default here.

A converged state on disk

`result.save(path)` writes the state and `SCFResult.load(path, system=...)` reads it back for `run_scf(starting_from` so a job that hits its wall clock does not cost the run. The system is supplied rather than stored — a resume already has the input file — and is checked against a fingerprint. `run_relax(checkpoint_dir=...)` does the same every ionic step *including the optimizer's own history*, which is the half that is easy to omit and most of the value: BFGS earns its convergence rate from the inverse Hessian and the trust radius, so a resume from positions alone takes its next step as if it were the first. Stopping a six-step relaxation at step two and resuming costs $2 + 4$ steps, not $2 + 6$.

Gamma-only storage: half the plane waves

At $k = 0$ a Kohn–Sham state can be chosen real, so $c(-G) = c^*(G)$ and one plane wave of each $(G, -G)$ pair carries all of it. `K_POINTS gamma` stores that half, which halves n_{pwx} and with it every array a band lives in — the wavefunctions, `vkb`, the Davidson subspace. On a 157-atom slab at $E_{\text{cut}} = 60$ Ry that is **189 GB against 96**. It is a memory feature, not a speed one: the two-bands-per-FFT packing of `vloc_psi_gamma` is deliberately not implemented.

Only the wavefunction sphere halves. The dense G set stays whole, where `pw.x` halves both. The memory is entirely in the plane-wave-sized arrays, so halving the dense set as well saves a fifth of a per cent and puts a conjugate fill inside every consumer of a real field. The consequence to know is that `ngm` here is not the one `pw.x` prints: the relation is $n_{gm} = 2n_{gm}^{\text{QE}} - 1$, and the test suite asserts it.

Three things carry the trick: the field is rebuilt from both halves, every sum over plane waves becomes $2\,\mathrm{Re}(\sum) - (G{=}0)$ — that vector being its own conjugate partner rather than half a pair — and $\mathrm{Im}\, c(0)$ must stay zero. **The last is the silent one**: a complex $c(0)$ makes the rebuilt field complex and the run converges to a plausible wrong answer.

The validation needs no other code. An explicit $k = 0$ on the whole sphere is the same calculation in the other storage, so the two agree to round-off: totals to **1e-14** Ry, eigenvalues to **3e-15** and forces to **3e-16** Ry/bohr, on LDA, PBE, LSDA and PBE+LSDA alike.

Refuses. `K_POINTS gamma` is *substituted* by an explicit $k = 0$ on the full sphere — the same physics at twice the storage, with a warning — for an ultrasoft or PAW dataset (`addusdens` and `newd` need their own `fact = 2`), for a run that uses symmetry (a rotation carries a stored G onto an unstored $-G'$, so the permutation would need a conjugation; `nosym = .true.` is the way out), for a spinor or spiral run, and for a **Hubbard** U. The last is the only one of them that would go wrong *during* the run rather than after it: n_σ is built from a plain $\langle\phi|\psi\rangle$ over the stored list, so the Hubbard potential and energy would follow occupations about a quarter of their true size and the SCF would converge silently to a different ground state. `estimate()` makes the same decision the driver makes, so it sizes the run either way.
Four things after the run refuse rather than substitute, because by then the states exist and the honest answer is to say the sum is not written: the projected density of states, the sum-over-states χ_0 (so an optical spectrum of a molecule in a box, which is the archetypal gamma case), an STM image and the vertical tunnelling transmission. Each is a plane-wave sum that would have to be $2\,\mathrm{Re}(\sum) - (G{=}0)$ and is not; the escape is the same explicit $k = 0$, which is an *exact* substitution rather than an approximate one.

The dynamical matrix works under gamma storage, including a partial one — the combination a molecule on a fixed slab needs. The Sternheimer solve is gamma-aware throughout (the projector, the CG's products, the preconditioner, the response density), and a displacement's source term is one jvp through `at_positions`, so it inherits the corrected `h_psi` with no branch of its own. Against the whole sphere: frequencies to **9.6e-10** cm^{-1} and the matrix to **1.5e-13** $\mathrm{Ry/bohr}^2$, with the acoustic residue identical; a partial matrix to **5.9e-11**. And χ_0 against a central difference of the density — which shares no machinery with either storage — to **7.0e-6**.
A **field** response is still refused, and the distinction is the reason: a field enters through the commutator $[H, \mathbf{r}]$, which builds plane-wave sums of its own. The dielectric tensor, the Raman tensor, the strain response and the Born charges all stand on it. It does not fail — with the guard lifted silicon's dielectric constant is $501.7/213.1/253.1$ against an isotropic 190.8 — which is why it is a refusal. A checkpoint refuses rather than drops a converged *magnetic field* (it is not the input field wherever `reducebf` changed it) and a *Hubbard setup* (without it `ns` is an array about nothing); `stress`, `solver` and `history` are deliberately not stored, so what comes back is a state and not a report.

2 Ground state

The Kohn–Sham problem, generalised because ultrasoft and PAW make the overlap nontrivial:

$$\hat{H}\,|\psi_{n\mathbf{k}}\rangle = \epsilon_{n\mathbf{k}}\,\hat{S}\,|\psi_{n\mathbf{k}}\rangle, \qquad \hat{S} = 1 + \sum_{I,ij} q_{ij}\,|\beta_i^I\rangle\langle\beta_j^I|$$

$$\hat{H} = -\nabla^2 + v_{\mathrm{loc}}(\mathbf{r}) + v_{\mathrm{H}}[n] + v_{xc}[n] \qquad + \sum_{I,ij} D_{ij}^I\,|\beta_i^I\rangle\langle\beta_j^I|$$

with $n(\mathbf{r}) = \sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} \big(|\psi_{n\mathbf{k}}|^2 + \sum_{ij} Q_{ij}^I \langle\psi|\beta_i\rangle\langle\beta_j|\psi\rangle\big)$, the augmentation term being what ultrasoft adds. D_{ij} is rebuilt from the potential every iteration, which is why the SCF is a fixed point in (n, D) and not in n alone. Convergence is measured as QE measures it — the Hartree energy of the residual:

$$dr^2 = \iint \frac{\delta n(\mathbf{r})\,\delta n(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|}\,d\mathbf{r}\,d\mathbf{r}' < \texttt{conv\_thr}$$

```
result = run_scf(system, pseudos, conv_thr=1.0e-10, mixing_mode="anderson")
print(result.total_energy, result.energy_terms)   # QE's own term-by-term split
print(result.fermi_energy, result.homo, result.lumo)
```

Metals work with every smearing QE has (`gaussian`, `marzari-vanderbilt`, `methfessel-paxton`, `fermi-dirac`) and with the tetrahedron family — both linear and Bloechl-corrected — usable as an occupation scheme *inside* the SCF, not only for a density of states.

Continuation across a change of spin regime. Give `run_scf` a previous result as `starting_from` — `System.with_spin` underneath — and it promotes a converged state into a target's variables: non-magnetic into collinear, collinear into noncollinear, spin-orbit switched on. It reaches the *same* solution as a fresh run ($\leq$4e-8 Ry on six cases) in as little as one iteration. The magnetization is **seeded** from the target's `starting_magnetization` when the source has none, because nothing in the SCF breaks spin symmetry on its own

and an unseeded promotion would converge back to the unpolarized solution and report success.

Convergence. `mixing_mode` selects `anderson` (default), `linear`, `broyden`, Kerker/Thomas–Fermi preconditioning (`TF`), or `local-TF`; `scf_solver` offers a Newton–Krylov alternative that is slower but reaches solutions the mixer cannot hold.

`TF` screens the residual with one Thomas–Fermi length for the whole cell. `local-TF` makes that length a function of $\rho(\mathbf{r})$, which is what a **slab** needs: the metal wants strong screening and the vacuum wants none, and a single compromise value over-screens one and under-screens the other — charge sloshing between the surfaces. The screened residual is the solution of

$$4\pi e^2 v(\mathbf{G}) + |\mathbf{G}|^2 (\alpha v)(\mathbf{G}) = |\mathbf{G}|^2 (\alpha\, \delta n)(\mathbf{G}), \qquad \alpha(\mathbf{r}) = 3\left(\tfrac{2\pi}{3}\right)^{5/3} r_s(\mathbf{r}),$$

with (αf) multiplied in *real* space, so the operator is not diagonal in $\mathbf{G}$ and is inverted iteratively at each step. On a cobalt film it converges at `mixing_beta = 0.7` where plain Anderson at the same β diverges and Kerker at 0.3 is still far out after 250 iterations. `max_iterations` defaults to 100 — **a hard SCF may need more**, and hitting the cap is reported as not converged rather than as an answer.

3 Bands, densities of states, projections

A band structure is one diagonalisation per $\mathbf{k}$ on a fixed density — `run_bands`, or `run_nscf` for a non-self-consistent run on a grid rather than a path. A density of states is

$$g(E) = \sum_{n\mathbf{k}} w_{\mathbf{k}}\, \delta(E - \epsilon_{n\mathbf{k}})$$

with δ replaced by a smearing function or evaluated by tetrahedron interpolation. The projected DOS weights each term by $\left|\langle \phi_\mu | \hat{S} | \psi_{n\mathbf{k}} \rangle\right|^2$, and the spilling parameter is what is *not* captured, $\sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} \big(1 - \sum_\mu |\langle \phi_\mu | \hat{S} | \psi \rangle|^2\big)$.

```python
from defumat.workflows.bands import run_bands
from defumat.workflows.dos import run_dos
from defumat.workflows.pdos import run_pdos

bands = run_bands(system, pseudos, scf.density, kpoints=path)   # a k-path

dos, _ = run_dos(system, pseudos, scf.density, grid=(12, 12, 12),
                 scheme="tetrahedra_opt")     # -> (DensityOfStates, NSCFResult)

pdos, _ = run_pdos(system, pseudos, scf)      # -> (ProjectedDOS, NSCFResult)
print(pdos.channels[0])                       # resolved by atom, l and m
print(pdos.charges.charges, pdos.charges.spilling)
```

The DOS schemes live behind a name registry (`DOS_SCHEMES`), and the projected DOS feeds *the same* registry as a per-band weight, so a PDOS and a DOS are the same integration. Projections are $\langle \phi | \hat{S} | \psi \rangle$ on Löwdin-orthogonalised pseudo-atomic orbitals. Validated against a purpose-built `projwfc.x` on seven cases: **6.9e-4** on a projection, **4.7e-5** on a Löwdin charge, and the spilling parameter to all four decimals it prints.

Resolved by j and m_j for a spin-orbit run

With `noncolin` and `lspinorb` the orbitals projected onto are the **spin-angle functions** $|l\,j\,m_j\rangle$ rather than $|l\,m\rangle$ times a spin, so a channel is labelled by j and m_j and the projection separates the two branches spin–orbit coupling splits a shell into. For platinum that is $5d_{3/2}$ against $5d_{5/2}$: one number, resolved, where an l-resolved projection reports their sum and cannot see the splitting at all. The functions are

$$|l\,j\,m_j\rangle \;=\; \sum_{\sigma=\uparrow,\downarrow} C\big(l,\tfrac{1}{2},j;\,m_j-\sigma,\sigma\big)\;Y_{l,\,m_j-\sigma}\;\chi_\sigma,$$

Clebsch–Gordan coefficients against the harmonics, and the number of columns is $\sum_{\text{shells}}(2j+1)$ — which is *not* in general twice the scalar count, because it is read off the dataset's own j channels.

```python
from defumat import Calculator

calc = Calculator.from_file("pt-soc-paw-nosym.in",   # noncolin + lspinorb
                            pseudo_dir="../pseudo")
pdos = calc.get_pdos()

print(pdos.channels[0])          # -> #1 Pt1 s_j0.5 m_j=-0.5
print(pdos.charges.format())     # -> s = 0.5032, p = 0.9944, d = 8.4903
print(pdos.charges.charges_lm)   # -> None: a spin-angle function has no m
```

Validated on fully-relativistic **ultrasoft and PAW** platinum datasets against `projwfc.x` runs generated for them, four ways. The *columns* — how many, and each one's (l, j, m_j) — match QE's own label table exactly. The two datasets differ there in a way worth seeing: the PAW file carries both j partners of every l, so its 18 columns are exactly twice the j-averaged 9, while the ultrasoft one has 12 against a scalar 11 — the count is $\sum(2j+1)$ read off the file and not a doubling. The *projections* agree to **6.3e-6**, compared as sums over each degenerate multiplet: a spinor band structure is Kramers-degenerate everywhere, and an individual $|\langle\phi|\hat{S}|\psi_n\rangle|^2$ is not invariant under the mixing inside a multiplet that either code's eigensolver is free to choose, where the sum is. The **Löwdin charges and the spilling parameter agree to every digit `projwfc.x` prints**. And the *curves* — the total and each shell's, on `projwfc.x`'s own energy grid and broadening — agree to **0.16 per cent of the peak** everywhere below the lowest empty band, above which the topmost states are the ones neither eigensolver converges. Timing, one core each from a converged ground state: `get_pdos` costs **1.65 s** against `projwfc.x`'s 0.62 s, and on both sides that is mostly rebuilding from the pseudopotential files rather than projecting — the projection and its integration are 0.137 s of it here.

Norm-conserving is untested rather than refused — no reference case is committed for it.

Two spin channels for a noncollinear run without spin–orbit coupling

With `noncolin` but `lspinorb = .false.` there is no j to resolve by: nothing couples s_z to the orbital motion, so s_z is still a good quantum number and the orbitals are $|l\,m\rangle\chi_\sigma$ — an up and a down copy of every harmonic, 2 natomwfc columns where a collinear run has natomwfc. What comes back is nonetheless *not* twice as many channels: the columns are routed into an **up and a down density of states**, which is `partialdos_nc`'s `nspin0 = 2` and is the shape an LSDA projection has. This is the regime of any magnetic run with scalar-

relativistic datasets, and it is what `h10-chain-noncolin.in` and `alas-magnetoelectric-nosoc.in` are.

```
from defumat import Calculator

calc = Calculator.from_file("h-atom-noncolin.in",   # noncolin, no lspinorb
                            pseudo_dir="../pseudo")
pdos = calc.get_pdos(symmetrize=False)

print(pdos.nspin)                # -> 2: an up channel and a down one
print(pdos.channels[0])          # -> #1 H1 s: no s_z, the spin is an axis now
print(pdos.charges.polarization) # -> [0.99857905]: the moment the s shell carries
```

The decomposition is along the **global** z and not along the local moment, exactly as QE's is: an in-plane texture reports two equal channels, which is a statement about the frame the orbitals are built in rather than a defect.

No `projwfc.x` reference for this regime is committed here, so what validates it is an identity instead, and a sharper one: without spin–orbit coupling a state polarized along $+z$ block-diagonalises the noncollinear Hamiltonian into the two collinear ones, so the projection of such a run must reproduce an LSDA run of the same cell channel for channel — and the LSDA route is itself validated against `projwfc.x`. On a hydrogen atom in a 12 bohr box the two runs agree to **3e-12 Ry** in total energy, their majority curves to **2e-5 of the peak** and their minority ones to **0.4 per cent**, and their Löwdin charges to **5e-3** on a polarization of **0.9986** electrons — which is the number a code that binned both spins into one channel, or split the weight evenly, would get wrong while passing every sum rule. The two bounds differ for an arithmetic reason rather than a chosen tolerance: the noncollinear branch reaches $\rho_\downarrow$ as $(n - |m|)/2$, a cancellation of two numbers of order 0.1 where the collinear branch carries $\rho_\downarrow$ itself, and that is worth 1.1×10^{-4} Ry on the *empty* minority eigenvalue where every occupied one agrees to 10^{-6}.

Unlike a spin–orbit run this case *does* have an m to report, so `charges_lm` is filled where a j-resolved projection leaves it `None`. `print_lowdin` allocates it only `IF (nspin /= 4)` and so prints none for either noncollinear branch; the reason it gives — a spin-angle function has no m — is true of the spin–orbit branch alone, and the table here is the richer one on purpose.

Refuses. A **symmetrised** spinor projection, in either noncollinear branch: averaging over the point group needs the SU(2) representation of each operation beside the rotation of the harmonics, and that is not built — run with `nosym` and the whole **k**-grid, where both codes use every point and neither averages, or pass `symmetrize=False`. A **fully-relativistic dataset with `lspinorb = .false.`**: QE dispatches the orbitals on `has_so` and their labels on `lspinorb`, so it builds j-resolved columns and calls them up/-down ones. An m-resolved charge is *absent* rather than refused for a **spin–orbit** run: `charges_lm` is `None` there, since a spin-angle function has an m_j and no m, and there is no p_x weight to report.

4 Pseudopotentials and functionals

Kinds: norm-conserving, **ultrasoft** and **PAW** — the two-grid split, the augmentation charge, the overlap operator, self-consistent D_{ij}, and PAW's one-centre terms including its radial Poisson solve and spherical quadrature. UPF v2 is read.

Functionals: LDA (PZ), **PBE**, **revPBE**, **PBEsol**, on the plane-wave grid and on the PAW spheres. The functional comes from the pseudopotential headers unless `input_dft` overrides it. Only the *energy* is written down; v_{xc}, and a GGA's v_1 and v_2, come from `jax.grad` of it.

Meta-GGA, potential-only branch: `input_dft = 'tb09'` (Tran–Blaha) and `'bj06'` (Becke–Johnson). These invert the rule above — they *are* potentials, there is no energy — so the consequences are enforced rather than documented: `run_scf` warns that its total is not the value of any functional it minimised, and forces, stress, phonons and response all refuse. Silicon's gap goes from LDA's 0.49 eV to **1.13 eV** against an experimental 1.17.

Van der Waals: Grimme **D2** (`vdw_corr = 'grimme-d2'`). Bilayer graphene matches `pw.x` to **3.1e-9 Ry** and binds at 3.23 Å where PBE alone has no minimum.

Refuses. UPF v1; an unimplemented functional (rather than silently substituting LDA); energy-carrying meta-GGAs (TPSS, SCAN, M06L), whose potential has a $\partial E/\partial\tau$ piece acting on the wavefunction that is not written; plain *ultrasoft* under a meta-GGA, which has no partial waves to reconstruct τ from inside the sphere where PAW does; EXX; and **D3, Tkatchenko–Scheffler, MBD and XDM** — where `pw.x` warns and silently runs with no correction at all.

5 Spin, magnetism and spin-orbit

regime	how to ask	note
unpolarised	default	
collinear	`nspin = 2`	one Fermi level, or two under `tot_magnetization`
noncollinear	`noncolin = .true.`	magnetization is a vector
spin-orbit	`+ lspinorb = .true.`	j-resolved projectors; NC, US and PAW

Three spin numbers are kept apart, and `System` exposes all three: `nspin` says which regime is in force, `npol` is the number of spinor components of a *wavefunction*, and `nspin_mag` the number of components of a *density*. They coincide at 1 and 2 and come apart at 4, where `npol = 2` but `nspin_mag` is **one** for a nonmagnetic spin-orbit run — which is why such a run costs about what a doubled unpolarized one costs.

Fixed occupations under `nspin = 2` fill each channel to its own count rather than searching for a level: `tot_magnetization` gives `nelup` and `neldw`, and `iweights_only` fills NINT($n_\uparrow$) bands in one channel and NINT($n_\downarrow$) in the other. That is the *only* shape the combination has — `pw.x` refuses fixed-occupation LSDA without a `tot_magnetization`, and requires an integer one — and this package makes the same two refusals at input, before anything is allocated. Each channel's highest occupied level is reported as `fermi_energy_up` / `fermi_energy_down` beside the overall `homo` and `lumo`, which is what `iweights` returns. Validated on an oxygen atom at `tot_magnetization = 2`, whose four-up / two-down filling is the Hund's-rule ground state: **4.9e-9 Ry** against `pw.x`.

Magnetic fields and constraints: a uniform field over the cell, a field inside one atom's sphere (through a `LOCAL_MAGNETIC_FIELDS` card that `pw.x` has no counterpart for), Elk's `reducebf`, and all four of QE's `constrained_magnetization` schemes. **The field's energy is not part of the reported total** — QE prints `etcon` and never adds it, and Elk excludes its external field's energy by the same convention.

Restarting an SCF from the middle (`checkpoint_dir`, `checkpoint_every`, `max_seconds`). A job killed at its wall clock used to lose every iteration it had run: a state reached disk only after `run_scf` returned. It now writes on a cadence and *resumes from the same directory*,

so a resubmitted batch script is the same command and continues.

```
from defumat import Calculator
calculator = Calculator.from_file("scf.in", checkpoint_dir="ckpt")
scf = calculator.get_scf()          # run again after a kill: it continues
```

A restart is three things, and the third is the one with no obvious home: the state (density, wavefunctions, `becsum`, `ns`, `tau`); the *mixer's history*, without which Anderson restarts empty and pays back the iterations the checkpoint saved; and the *loop state* — the iteration number, `dr2` and the diagonalisation threshold `ethr`. That last one matters because QE's threshold schedule is indexed on the iteration number, so a resume re-entering at iteration 1 with a fresh threshold converges on a different schedule than the one it left. With all three carried the restart is exact: on silicon at `conv_thr = 1e-12`, stopping and resuming costs the *same* total iteration count as an uninterrupted run at three different mixing parameters (17/17, 13/13, 11/11), with energies agreeing to 2×10^{-15} Ry. Both Quantum ESPRESSO and Elk restart this way, and carry the same three parts.

`max_seconds` lets the loop stop *itself* before the scheduler does, writing a checkpoint on the way out, so a wall-clock kill lands after a checkpoint rather than between two. It is checked between iterations, so set it at least one iteration short of the real limit.

Refused. Checkpointing is switched off, once and with a message, for a run whose field changes between iterations — Elk's `reducebf` and the fixed-spin-moment feedback. The state's field is then not the input's, and `save_state` will not write a lossy one. `checkpoint_every` must be at least 1.
Know the cost. The payload is the wavefunctions, which is tens of gigabytes on a large cell, so this is a scratch-directory operation and the cadence is a knob rather than every iteration. It has not been timed on a large cell; the default of 10 is a guess until it is.

A starting magnetic texture (`STARTING_MOMENTS` card): one starting moment per *atom*, cartesian, in Bohr magnetons, one line per atom in the order of `ATOMIC_POSITIONS`. QE's `starting_magnetization`, `angle1` and `angle2` are per *species*, so a helix, a cycloid or a skyrmion cannot be stated in a `pw.x` input at all.

It decides three things, and the first is why it matters more than a starting guess usually does. It decides the **symmetry group**; it decides whether the run is **magnetic at all** (a noncollinear input whose texture is given only through this card is a magnetic run, with a four-component density — this used to be read off the per-species `starting_magnetization` alone, so such an input converged, reported a total energy and never had a magnetization); and it seeds the **starting density**, one moment per atom rather than one per species. The magnetic group is filtered by the per-atom moments $\mathbf{m}_i$ — an operation survives only if it maps that axial vector field onto itself, possibly followed by time reversal — and it is built from the input, once, never from the converged state. Built from per-species values it is a *ferromagnet's* group, which keeps operations a texture does not have, and `sym_rho` then averages the texture away a few iterations in while the charge converges and `dr2` reports nothing.

A per-atom *applied* field is filtered with the moments, together, under one choice of time reversal, exactly as Elk's `findsym.f90` tests `bfcmt0`. So a `LOCAL_MAGNETIC_FIELDS` card that asks for a texture now cuts the group by itself and needs no card of its own.

```
STARTING_MOMENTS
   0.00000000   1.50000000   0.00000000
  −0.75000000   1.29903811   0.00000000
  −1.29903811   0.75000000   0.00000000
```

Holding a texture (`constrained_magnetization = 'atomic texture'`) constrains the *full* unit vector of every atom's moment,

$$E_{\mathrm{con}} = \lambda \sum_i \left(1 - \frac{\mathbf{m}_i \cdot \hat{\mathbf{n}}_i}{|\mathbf{m}_i|}\right),$$

with $\hat{\mathbf{n}}_i$ the normalised `STARTING_MOMENTS` rows. It is not one of QE's schemes and has no `i_cons` number. QE's nearest, `'atomic direction'` (`i_cons = 2`), constrains $m_z/|m|$ — the polar angle *alone* — so it is exactly satisfied by every texture lying in a plane that contains z, a helix and a collinear state alike, at zero penalty: it cannot hold one, and reports success while failing to.

Refused. `'atomic texture'` needs `noncolin = .true.` (a collinear moment has no direction beyond its sign) and a `STARTING_MOMENTS` card with no zero row (a zero vector carries no direction). `STARTING_MOMENTS` itself needs `noncolin` for any x or y component, the same rule `B_field` and `LOCAL_MAGNETIC_FIELDS` are held to, and one row per atom.
Checked, not remembered: the card above parses through `build_system`, reaches `system.starting_moments` unchanged, becomes `m_loc` verbatim, and `constraint_targets` returns it normalised — three unit rows. The starting density built from it carries opposite moment lobes on two antiparallel sites and integrates to zero over the cell.
One thing it does not yet reach, for a PAW dataset: the atomic `becsum`, which is still split by the per-*species* `starting_magnetization`. So on PAW the charge's starting texture and the one-centre starting occupations disagree at iteration 1. The SCF repairs it and nothing downstream is wrong at convergence; it costs iterations, and it is written down here rather than left to be met.
Not covered, and this one is a sentence rather than a `raise`: a textured starting *density* handed to `run_scf` by the caller. The symmetry group is decided from the input and never from the density, so a texture that reaches the run only that way is invisible to the filter and will be symmetrised away. Give the same texture as a `STARTING_MOMENTS` card, or set `nosym = .true.`

DFT+U: both rotationally-invariant functionals, on `atomic`, `ortho-atomic` and `norm-atomic` projectors, read from the `HUBBARD` card. Forces come from differentiating through projectors that move with the atoms.

The card selects which functional runs, exactly as `pw.x` does it — the namelist's `lda_plus_u_kind` is obsolete there and warned about. A card carrying only U, J_0, `ALPHA` and `BETA` is Dudarev's simplified functional; any J, B, E_2 or E_3 selects Liechtenstein's full one, whose interaction is the whole Coulomb matrix

$$\mathrm{vee}(m_1 m_2 m_3 m_4) = \sum_k F^k \frac{4\pi}{2k+1} \sum_{q=-k}^{k} a_{kq}(m_1 m_3)\, a_{kq}(m_2 m_4)$$

rather than one number, with the Slater integrals F^k fixed by U and J. Against `pw.x` on antiferromagnetic FeO: **4.2e-9 Ry** at $J = 1$ eV. *Forces work for both*, and for the full functional `pw.x` does not have them at all (`force_hub.f90` stops on "forces in the DFT+U+J scheme are not implemented"); a central difference of the SCF energy agrees to 1.3e-4 Ry/bohr. **Seed that difference from the undisplaced run** — with a full interaction matrix the DFT+U landscape has more than one solution, and cold-started displacements land on a different one, which does not look like an error, it looks like a force.

```
from defumat import Calculator
scf = Calculator.from_file("feo.in").get_scf()
print(scf.energy_terms["hubbard"])
```

```
HUBBARD {atomic}
U Fe1-3d 4.3
J Fe1-3d 1.0     # selects the full functional
```

Around-mean-field double counting (`hubbard_double_counting = 'amf'`) is the alternative to the fully-localised limit that both QE's functionals assume, and `pw.x` has neither the option nor the code. FLL treats the occupations as integers, which is right for a localised magnetic insulator and wrong for a metal; AMF subtracts the shell's mean occupation before the interaction, so a uniformly filled shell is corrected by *exactly* nothing — 0 to 10^{-14} at any filling, where FLL is not. On FeO its Hubbard energy is -0.005 Ry against FLL's $+0.232$. It needs the full functional, there being no interaction matrix in the simplified one to apply the shift to.

Slater integrals from the orbital (`hubbard_slater = 'yukawa'`) computes the interaction instead of parameterising it: F^k is the double radial integral of the manifold's own all-electron partial wave against a screened Coulomb kernel,

$$F^k = \iint \mathrm{d}r\,\mathrm{d}r'\;[r\phi(r)]^2[r'\phi(r')]^2\,(2k+1)\,\lambda\,i_k(\lambda r_<)\,\tilde{k}_k(\lambda r_>),$$

which becomes the bare $r_<^k/r_>^{k+1}$ as $\lambda \to 0$. Give λ on the card (`LAMBDA Ni-3d 2.1`) or give a U and let the screening length be solved for — either way F^0, F^2, F^4 and J all come out of the orbital. Nickel's unscreened 3d gives $F^4/F^2 = 0.626$ against the 0.625 that QE and Elk both hardcode, and $J = 1.28$ eV.

```
&system
   hubbard_slater = 'yukawa'
/
HUBBARD {ortho-atomic}
U Ni-3d 5.0      # lambda is solved for; F2, F4 and J follow
```

On a spinor (`noncolin`, with or without `lspinorb`) the occupation matrix is one 2×2 matrix in spin space rather than two real matrices, measured on projector columns that are themselves spinors, and its off-diagonal blocks are what let the shell's moment point anywhere. Both functionals work there. Against `pw.x`: **7.7e-9 Ry** on relativistic BN and **1.4e-9 Ry** on fully-relativistic iron carrying $U = 2.20$, $J = 1.75$ eV with its moment turned into the xy plane, with $\mathrm{Tr}[n^{\sigma\sigma}]$ matching every digit `write_ns` prints.

Tensor moments put that matrix in a basis where each element is a multipole rather than a number: an orthonormal, complete set Γ_t^{kpr} with k the rank in the orbital index, p the rank in spin and r their coupling, so that

$$n = \sum_{kprt} w_t^{kpr}\,\Gamma_t^{kpr},$$

with real w. The charge is w^{000}, the spin moment is w^{011}, and $\mathbf{L}\cdot\mathbf{S}$ is w_0^{110} alone — to 10^{-11} over all one hundred moments of a d shell.

Fixing one (a `TENSOR_MOMENTS` card) converges the field to a state carrying it, which selects an orbital or multipolar ordering the field would not find on its own. The penalty is written down and its potential is `jax.grad` of it, and, like a constrained moment's, its energy is not part of the reported total — so the number that comes back is the unconstrained functional at the constrained state.

```
&system
   tensor_moment_penalty = 20.0
/
```

```
TENSOR_MOMENTS
  N-2p  1  1  0  0  -0.40        # hold L.S at -0.40
```

Being quadratic, the penalty settles where its gradient balances the material's own restoring force, so the moment *approaches* its target as the penalty stiffens rather than reaching it: on nitrogen's $2p$, whose free $\mathbf{L}\cdot\mathbf{S}$ is -0.0006, holding it at -0.40 gives -0.109, -0.263, -0.355 and -0.388 at $\lambda = 1, 5, 20, 80$, with the energy rising monotonically by 2.6, 15.3, 27.4 and 32.6 mRy.

Refuses. A spinor occupation matrix needs `nosym`: its group average takes the $SU(2)$ representation of each operation beside the rotation of the m indices, and averaging the m indices alone leaves the moment pointing somewhere else. A fixed tensor moment needs a spinor: for a collinear run the spin-rank moments keep only their z component and the constraint degenerates into the fixed spin moment `constrained_magnetization` already is. `hubbard_slater = 'yukawa'` needs a **PAW** dataset, and this is a measurement rather than a preference: only PAW carries the all-electron partial wave. A norm-conserving χ is normalised and gives a pseudo-orbital F^0 far too small (silicon's 3p: 8.8 eV), and an *ultrasoft* χ is not even normalised — iron's 3d has $\int(r\chi)^2 = 0.41$, the rest of the norm living in the augmentation charge, and its F^0 comes out at 2.1 eV where the answer is above twenty. The `norm` that comes back with each manifold is what says whether it is bound inside the cutoff, and the cutoff defaults to the dataset's own augmentation radius because a partial wave *diverges* outside it. Elk's `inpdftu = 2` and `3` (Slater and Racah *input*) have no card syntax: they are a change of coordinates on the three numbers (U, J, B) already spans, and QE's card has spent the names E_2 and E_3 already. The FLL/AMF interpolation (Elk's `dftu = 3`) is not here because it is not in Elk either any more — documented in the 11.0.2 manual, removed from the 11.0.2 source.

Refuses. The intersite V, background channels, the orbital-resolved variant, the `wf`/`pseudo` projector sets, and noncollinear `ns_nc`; and PAW + GGA + a noncollinear magnetization together.
A **magnetic field in a noncollinear run with no starting moment** is refused rather than run. `domag` comes from `starting_magnetization` and from nothing else (`setup.f90`), so such a run has `nspin_mag = 1` and a density with no magnetization channels for the field to act on. Set `starting_magnetization` for one species — any nonzero value will do, the field decides where the moment goes. Worth knowing: `pw.x` *accepts* this input and silently drops the field, because `add_bfield` writes it into the potential's magnetization channels and `vloc_psi_nc` applies those only `IF (domag)`.
The **residual solver** cannot take a fixed occupation whose filled/empty boundary cuts a **degenerate multiplet** — which is exactly what an atom's Hund's-rule filling does. Which member of the multiplet the eigensolver returns is arbitrary, so the density map is not a function of the density and a Newton step has nothing to converge on; it is diagnosed by name rather than left as a `NaN`. The mixer damps its way past it, and a *gapped* fixed-occupation LSDA cell agrees between the two solvers to 5e-12 Ry.

6 Forces, stress and relaxation

$$\mathbf{F}_I = -\left.\frac{\partial E}{\partial \boldsymbol{\tau}_I}\right|_{\psi \text{ fixed}}, \qquad \sigma_{ij} = -\frac{1}{\Omega}\left.\frac{\partial E}{\partial \varepsilon_{ij}}\right|_{\psi \text{ fixed}}$$

Both are one gradient. With ultrasoft the orthonormality constraint $\langle\psi_n|\hat{S}|\psi_m\rangle = \delta_{nm}$ is

carried explicitly, so its multiplier term — QE's Pulay contribution — falls out of the same derivative rather than being added afterwards. A variable-cell relaxation minimises the **enthalpy**, which is why a relaxed crystal carries the applied pressure rather than having zero stress:

$$H = E + P\,\Omega, \qquad \frac{\partial H}{\partial h} = \Omega\,(P\,\mathbf{1} - \sigma)\,h^{-T}$$

```python
from defumat.forces import compute_forces
from defumat.stress import compute_stress
from defumat.workflows.relax import run_relax
from defumat.workflows.vc_relax import run_vc_relax

forces = compute_forces(calculation, result)      # method=... for QE's own terms
print(forces.forces)                              # (nat, 3) in Ry/bohr

stress = compute_stress(calculation, result, terms=True)
print(stress.pressure)                            # kbar

relaxed = run_relax(system, pseudos, forc_conv_thr=1.0e-4)
cell = run_vc_relax(system, pseudos, press=500.0) # kbar
print(cell.pulay_error)   # the frozen-basis error, reported not hidden
```

QE's `force_lc`, `force_cc`, `force_ew`, `force_us`, `addusforce` and `force_corr` are transcribed too, behind the same registry — **as a cross-check, not as the implementation**. The two share no machinery, which is what makes disagreement informative; it is how the augmentation force's sign and a missing gradient correction were found.

For a noncollinear or spin-orbit run the same expression is evaluated on two-component spinors: the nonlocal term takes the pseudopotential's bare `dvan_so` (a complex 2×2 matrix in spin space) and the orthonormality constraint takes `qq_so`, and the state is one coefficient vector of length $2\,n_{\mathrm{pw}}$ rather than two. Nothing else in the functional changes.

```python
from defumat import Calculator

calculator = Calculator.from_file("pt2-soc-force.in")   # noncolin, lspinorb
forces = calculator.get_forces()          # (nat, 3) Ry/bohr
stress = calculator.get_stress()          # (3, 3)   Ry/bohr^3
print(forces.max_force, stress.pressure_kbar)
```

Validated against `pw.x` with `tprnfor`/`tstress` on a four-atom noncollinear hydrogen chain (norm-conserving) to 8.9×10^{-7} Ry/bohr, doubled fcc platinum with spin-orbit coupling to 7.5×10^{-6} (ultrasoft) and 7.3×10^{-7} (PAW), and the stress on those three plus QE's own three `pw_spinorbit` cases to $\leq 1.2\times10^{-6}$ Ry/bohr^3; and, without any Fortran at all, against a central finite difference of the frozen energy to 6.2×10^{-9} Ry/bohr.

A noncollinear or spin-orbit run has forces, a stress and a relaxation, but only through the *autodiff* method: `method='analytic'` is refused by name, because `force_us`/`stres_knl` are transcriptions of QE's hand-derived expressions and have no spinor form. The force on an atom of a **spin spiral** is refused separately — its two spinor components live on different plane-wave spheres, so the nonlocal term needs the projectors of both; `dE/dq` is what a spiral has instead. **Elastic constants and electrostriction** stay refused for `noncolin`: they differentiate the same functional a third time and their first-order wavefunctions come from a Sternheimer solve that has no spinor form.

7 Response: dielectric, phonons, Raman

The **velocity operator** comes first and everything else uses it: it is one `jvp` of $\hat{H}(\mathbf{k})$ at a frozen sphere, because $\partial\hat{H}/\partial k_\alpha = i[\hat{H}, r_\alpha]$ in the periodic gauge. `band_velocities` exposes it — `Calculator.get_band_velocities` is the short way — and because the overlap carries a velocity too, a band velocity is $\langle\psi|\partial_k\hat{H} - \epsilon\,\partial_k\hat{S}|\psi\rangle$.

```
calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")

velocities = calc.get_band_velocities()          # .velocities is (nk, nbnd, 3)
elsewhere = calc.get_band_velocities(kpoints=path)
```

Every first-order quantity then comes from the Sternheimer equation — the response of an occupied orbital to a perturbation, projected outside the occupied manifold so that no empty states and no energy denominators appear:

$$\left(\hat{H} - \epsilon_n\hat{S} + \alpha\hat{Q}\right)|\Delta\psi_n\rangle = -\,\hat{P}_c^{+}\,\Delta V\,|\psi_n\rangle$$

It is solved self-consistently, because ΔV contains the response of the potential to the response of the density,

$$\Delta V_{\mathrm{scf}} = \Delta V_{\mathrm{ext}} + \int K(\mathbf{r},\mathbf{r}')\,\Delta n(\mathbf{r}')\,d\mathbf{r}'$$

and the kernel K is one `jvp` of `v_of_rho` rather than a second implementation. From its solutions:

$$\epsilon^{\infty}_{\alpha\beta} = \delta_{\alpha\beta} + \frac{8\pi}{\Omega}\sum_{n\mathbf{k}} w_{\mathbf{k}}\langle\Delta_\alpha\psi_n|\partial_\beta\psi_n\rangle, \qquad Z^{*}_{I,\alpha\beta} = \frac{\partial F_{I\alpha}}{\partial\mathcal{E}_\beta}$$

$$C_{I\alpha,J\beta} = \frac{\partial^2 E}{\partial\tau_{I\alpha}\partial\tau_{J\beta}}, \qquad \omega^2\mathbf{e} = \frac{1}{\sqrt{M_I M_J}}\,C\,\mathbf{e}, \qquad \frac{\partial\epsilon_{ij}}{\partial\tau_{I\gamma}}\ \text{(Raman)}$$

The Raman tensor is the second-order energy differentiated once more along a displacement, at *frozen first-order* wavefunctions — the $2n{+}1$ theorem, which is why a third derivative costs one more `jvp` and not a new solve.

```
from defumat.response import (dielectric_tensor, dynamical_matrix,
                              raman_tensors, vibrational_spectrum)

eps = dielectric_tensor(calculation, scf.wavefunctions, scf.eigenvalues,
                        scf.density)
print(eps.epsilon)                       # 3x3, cartesian

ph = dynamical_matrix(calculation, scf.wavefunctions, scf.eigenvalues,
                      scf.density, scf.becsum)
print(ph.frequencies)                    # cm^-1

raman = raman_tensors(calculation, scf, born_charges=True, keep_internals=True)
spectrum = vibrational_spectrum(calculation, scf)   # per-mode Raman and IR
print(spectrum.raman_activity, spectrum.depolarisation)
```

`born_effective_charges` gives Z^* on its own, and `mode_activities` is the bare assembly — a transcription of `dynmat.x`'s `RamanIR`, kept a pure function of arrays so it is testable without a self-consistent run.

Ultrasoft and PAW work for the dielectric constant ($\leq$1.2e-4 against `ph.x` on four cases) *and for the dynamical matrix* — silicon's optical mode comes out at **513.295** and **513.378** cm^{-1} against `ph.x`'s 513.275 and 513.404, which is tighter than the norm-conserving case's 0.05. Four things make the difference and all four switch themselves off when S is the identity: the source term is $(\mathrm{d}H/\mathrm{d}u - \varepsilon\,\mathrm{d}S/\mathrm{d}u)|\psi\rangle$; the first-order state has an occupied block the Sternheimer solve does not produce; the mixed state changes at *frozen* states, because the augmentation charge and the projectors travel with their atom; and the orthonormality multipliers are variables of the functional that move — as a **matrix**, since a diagonal one is not invariant under the occupied-manifold rotation the state tangent is free in. Metals work for the response and for a norm-conserving dynamical matrix; and a symmetry-reduced wedge works as of the rank-n symmetriser.

The Raman tensor is ultrasoft and PAW as well, at **1.2e-4** on both against a central difference of ϵ over re-converged displaced cells, where the norm-conserving control on the same script is 6.8e-4. It took **two tangents that are only right together**, and either alone is worse than neither: the state tangent is $P_c\,\mathrm{d}\psi + \psi^{\mathrm{ort}}$, the occupied block again; and $|b\rangle = P_c\,\mathbf{r}|\psi\rangle$ is *not* the solution of its own linear equation once `adddvepsi_us` has applied S to it and added the augmentation dipole, so $\mathrm{d}b$ is the tangent of a *composition* rather than of a solve. What located them is that $\partial\epsilon/\partial\tau$ is a sum of five partial derivatives and each one can be finite-differenced on its own against a re-converged run: three agreed to 7e-4 and two did not.

A subset of the atoms can be displaced on its own. The cost of a dynamical matrix is that $3\,n_{\mathrm{at}}$ bare perturbations and $3\,n_{\mathrm{at}}$ first-order wavefunctions are held at once, which is what stops it running on a large cell; `atoms=` solves only the perturbations asked for and returns that subset's own block, diagonalised with its own masses. That is the frozen-substrate approximation — a molecule on a fixed slab — and the modes it leaves are the frustrated translations and rotations, which are the physics of such a calculation rather than an artefact. A `on_row` callback receives each row as it is assembled, so a run that outlives its job keeps what it paid for.

```
phonons = calc.get_phonons(atoms=range(57))    # the molecule, slab held fixed
phonons.frequencies                             # (3 * 57,) in cm^-1
```

It works on norm-conserving, ultrasoft and PAW datasets. The validation needs no reference of any kind, which is unusual here: a partial dynamical matrix is not an approximation — it solves fewer perturbations of the *same* equations — so the whole-cell run is exact for what the two share, and the subset's rows reproduce it to **9.3e-15** (norm-conserving), **2.8e-14** (ultrasoft) and **4.9e-15** (PAW) Ry/bohr^2. The physical check is that freezing one of silicon's two atoms turns the optical mode's reduced mass $m/2$ into m, so the frequency falls by $\sqrt{2}$: $519.205 \to 367.146$, a ratio of 1.414173.

A soft mode converges in `ecutrho`, and the finite-difference route hides it. The analytic Hessian and a finite difference of the forces are the same quantity in exact arithmetic and are not on a grid: the FFT box breaks continuous translation invariance, so the energy carries a small ripple as an atom moves across it (the "egg-box" effect). The analytic second derivative takes the *point* curvature of that ripple; a finite difference averages it over the displacement. The two therefore differ by the amount the dense grid is not translationally invariant — and because that error is roughly a fixed curvature, it matters in proportion to how *small* the mode's own curvature is. A stiff mode barely notices; a soft one can be several per cent out.

It is `ecutrho` and not `ecutwfc` that fixes it, the ripple living on the dense grid where the density and the local potential do. Measured on N_2 in a 12 bohr cube (SG15 ONCV PBE, `nosym`, `K_POINTS gamma`, one atom frozen, the other at 1.15 Å), with the finite difference taken

as a central difference of the forces at ± 0.03 Å:

	transverse (cm^{-1})	gap	stretch (cm^{-1})
40/160 Ry, analytic	314.12		1483.49
40/160 Ry, finite difference	296.59	17.5	1490.77
40/320 Ry, analytic	306.35		1488.1
40/320 Ry, finite difference	301.9	4.4	1495.2

Raising *only* `ecutrho` moves the frustrated-rotation pair by 8 cm^{-1} analytically and 5 cm^{-1} by finite difference, closing their gap from **17.5 to 4.4** cm^{-1}, while the stretch is 0.5% off either way at both cutoffs — which is the signature. The analytic asymmetry is 3e-8 and 1.4e-8 Ry/bohr^2, so neither number is a convergence failure of the solve.

What makes this worth stating rather than discovering: both routes converge, each agrees with itself, and they drift *together*, so nothing in either calculation announces the error. The practical rule is that a soft mode wants its own `ecutrho` convergence pair, and the shift between them is the error bar to quote — not the agreement between the analytic and finite-difference routes at one cutoff, which is a weaker statement than it looks. (Measurements contributed from a production study of a molecule on a surface, where the modes of interest are 10–15 meV.)

A trap worth knowing: a response on a reduced k-set is a *polar vector field* and must be symmetrised as one. Running the whole grid instead is sound only if the grid is closed under the point group — a **shifted** Monkhorst–Pack grid is not.

7.1 Phonons away from Γ

A displacement pattern $\mathbf{u}_s(\mathbf{R}) = \mathbf{u}_s e^{i\mathbf{q}\cdot\mathbf{R}}$ is not a displacement of the unit cell — it is periodic only on a supercell commensurate with $\mathbf{q}$, which is exactly what a frozen-phonon calculation pays for. Linear response does not pay it, because the *first-order* quantities are still lattice periodic once their $e^{i\mathbf{q}\cdot\mathbf{r}}$ is taken out. The perturbation is $e^{i\mathbf{q}\cdot\mathbf{r}}$ times a periodic function, so $\Delta V_{\mathbf{q}}|\psi_{\mathbf{k}}\rangle$ is a Bloch state at $\mathbf{k}+\mathbf{q}$: the calculation stays in the unit cell, with one extra plane-wave sphere.

That asymmetry is the whole of it. The operator, the projector and the preconditioner belong to $\mathbf{k}+\mathbf{q}$; the eigenvalue subtracted is the one of the band being perturbed, at $\mathbf{k}$:

$$\left(\hat{H}_{\mathbf{k}+\mathbf{q}} - \epsilon_{n\mathbf{k}}\hat{S}_{\mathbf{k}+\mathbf{q}} + \alpha\hat{Q}\right)|\Delta\psi_{n\mathbf{k}}\rangle = -\,\hat{P}_c^{\mathbf{k}+\mathbf{q}}\,\Delta V_{\mathbf{q}}\,|\psi_{n\mathbf{k}}\rangle$$

and the response density is $\Delta n_{\mathbf{q}}(\mathbf{r}) = (2/\Omega)\sum_{n\mathbf{k}} w_{\mathbf{k}}\,\psi^*_{n\mathbf{k}}\Delta\psi_{n\mathbf{k}}$, which is complex — at Γ it collapses to twice a real part and nowhere else.

```
# nosym = .true., noinv = .true., on an unshifted grid -- see the box below
calc = Calculator.from_file("si-nosym.in", pseudo_dir="pseudo")

ph = calc.get_phonons_at_q(q=(0.5, 0.5, 0.5))          # crystal coordinates, L
print(ph.frequencies)                                   # cm^-1
print(ph.matrix.shape)                                  # (nat, 3, nat, 3), complex

xpoint = calc.get_phonons_at_q(q=(0.0, -1.0, 0.0), q_cartesian=True)
```

`q` is in crystal coordinates of the reciprocal lattice unless `q_cartesian` is set, which reads it in the $2\pi/a$ units `ph.x` prints. The matrix is **complex**: $D(\mathbf{q})$ is hermitian rather than

symmetric, and only at $\mathbf{q} = 0$ — or where the crystal has inversion — does it collapse to a real one.

Silicon on a 64-point unshifted grid, against the vendored `ph.x`: at L the six modes are 101.789, 101.789, 382.235, 405.099, 488.028 and 488.028 cm^{-1} against 101.843, 101.843, 382.241, 405.121, 488.019 and 488.019, the worst of them **0.054** cm^{-1}; at X the worst is **0.038**. That is the same floor the zone centre has and for the same reason (QE interpolates its radial form factors from a $\mathrm{d}q = 0.01$ table where this integrates them directly). `ph.x` runs the eight-point wedge with its response symmetrised where this runs the whole grid with nothing symmetrised, so the comparison is also the only check the two routes get against each other.

Only the Ewald sum knows about q, among the terms that do not involve the response. The second derivative of the external potential at frozen density is diagonal in the atom index — both derivatives are taken at the same site, so the two modulation phases cancel — and it is therefore the same matrix at every wavevector. The ion–ion sum is not diagonal, and it is the one piece rebuilt.

Refuses. A symmetry-reduced k-set: a response has to be averaged over the operations that leave the *perturbation* invariant, and for a phonon at **q** those are the ones with $S\mathbf{q} = \mathbf{q} + \mathbf{G}$ — the small group of **q**, which is not the crystal's group and is not written. Run with `nosym` and `noinv` on an **unshifted** grid, which is the grid that is closed under the point group. There is no **dispersion** either: that needs the star of **q** and `q2r`/`matdyn`'s Fourier interpolation on top of this. Ultrasoft and PAW are refused (S moves with the atoms, and the orthonormality multipliers carry a term whose bra is at **k** and whose ket is at $\mathbf{k} + \mathbf{q}$, where q_{ij} pairs projectors on one sphere); so are metals, spin polarization, spinors, spin spirals, meta-GGA and DFT+U. A **nonlinear core correction** is refused for a reason that belongs to **q** rather than to the dataset: two atoms' core charges overlap inside a nonlinear E_{xc}, so unlike every other frozen term that one *is* a function of **q**.

Refuses. Born charges for PAW (`int3_paw` against `becsumort` is missing, and without it the expression is wrong in sign as well as size); $\chi^{(2)}$ and the electro-optic tensor (the second-order source term has nothing to build the $\langle u_i | r_k | u_j \rangle$ piece from — and it is **42%** of the answer, measured); a partial dynamical matrix (`atoms=`) of a *symmetrised* run, since `symdvscf` acts on the $3\,n_{\mathrm{at}}$ stack as one object — an operation rotates the cartesian direction *and* permutes the atom, so it mixes a mode inside the subset with one outside it, and such a run wants `nosym`; the dynamical matrix of an ultrasoft or PAW *metal* (the `wg`/`wk` weight split was derived for a response whose `becsum` dependence is entirely inside `dpsi`, and an insulator cannot say which weight the three further tangents belong with); any response under a potential-only meta-GGA; and a shifted grid with a reduced k-set.
Also refused: a ground state converged under a **magnetic field or a constrained moment**. The whole stack rebuilds its potential from the field the *input* asked for, and that is not the field the density belongs to — `reducebf` scales a field away as the run proceeds (7% of its input value after 25 iterations at 0.9) and the fixed-spin-moment scheme replaces it outright, which is why `SCFResult` carries `magnetic_field` and `field_scale` separately. Threading that pair through is the missing piece for a field put in by hand; a `constrained_magnetization = 'fsm'` moment needs the *induced* field as well, and its field comes from a feedback update rather than from a penalty, so it is not a derivative of anything. Re-run the ground state without the field.

LO–TO splitting and the static dielectric constant

At Γ a polar crystal's dynamical matrix is not analytic. An optical mode that moves the ions against each other builds a macroscopic electric field, and what the field costs depends on the *direction* the zone centre is approached from, so the matrix acquires a rank-one term:

$$D_{I\alpha,J\beta} \;\to\; D_{I\alpha,J\beta} + \frac{4\pi e^2}{\Omega}\,\frac{(\mathbf{q}\cdot Z^*_I)_\alpha\,(\mathbf{q}\cdot Z^*_J)_\beta}{\mathbf{q}\cdot\epsilon^\infty\cdot\mathbf{q}}$$

It raises the longitudinal mode and leaves the transverse ones alone — the LO–TO splitting — and it vanishes identically where Z^* does, so a non-polar crystal is untouched. Contracting the same two ingredients the other way gives the ionic screening of a *static* field, one oscillator per polar mode:

$$\epsilon^0_{ij} = \epsilon^\infty_{ij} + \frac{4\pi e^2}{\Omega}\sum_\nu \frac{p^\nu_i\,p^\nu_j}{\omega^2_\nu}, \qquad p^\nu_i = \sum_{I\gamma} Z^*_{I,i\gamma}\,z^\nu_{I\gamma}$$

where p is the same mode dipole the infrared activity is the square of. So ϵ^0 and ϵ^∞ are two ends of one contraction, and together with the frequencies they satisfy the Lyddane–Sachs–Teller relation $\epsilon^0/\epsilon^\infty = (\omega_{\mathrm{LO}}/\omega_{\mathrm{TO}})^2$.

```
spectrum = calc.get_vibrational_spectrum(neutralize=True)
print(spectrum.static_permittivity)          # eps_0, 3x3
print(spectrum.ionic_permittivity)           # the polar modes' share of it

lo = calc.get_vibrational_spectrum(loto_direction=(1, 0, 0), neutralize=True)
print(lo.frequencies)                        # the LO modes along x
```

`nonanal` adds the term to a dynamical matrix on its own and `loto_modes` diagonalises the result; `polar_mode_permittivity` is the oscillator sum, and `neutral_born_charges` the sum-rule correction below. All four are pure functions of arrays, so they can be checked without a self-consistent run behind them.

Both halves agree with `dynmat.x` — which applies the same term and the same oscillator sum to the tensors this code writes to its `fildyn` — to every digit it prints. **The Lyddane–Sachs–Teller relation is the check neither half can pass alone**, since one side is a ratio of two diagonalisations and the other a sum of dipoles over ω^2; on AlAs it holds to **5.0e-11**.

It needs a charge-neutral Z^* and that is worth knowing before reading an LO frequency. The sum rule $\sum_I Z^*_I = 0$ says a rigid translation builds no field; a computed Z^* misses it by the basis-set error, which *charges the crystal* — AlAs at `ecutwfc = 10` misses it by -1.257, and the non-analytic term then lifts a longitudinal *acoustic* mode from 1.8 to 33.8 cm^{-1} and puts the LO frequency 7.7 cm^{-1} low. `neutralize=True` shares the violation out over the atoms, which is what `dynmat.x` does under any `asr` but `'no'`. Comparing against another code does not see this: it is handed the same charges and reproduces the same wrong number. LST does.

Refuses. Everything the Raman tensors and the Born charges refuse, since it is built from both: PAW Z^*, metals, and the rest of the list above. The splitting is a Γ quantity by construction — it is what $q \to 0$ leaves behind — so a dispersion through it needs phonons at $q \neq 0$, which are not implemented.

Elastic constants, electrostriction and the elasto-optic tensor

The same machinery in the *strain* coordinate rather than the atomic one. **The strain *response* itself works on all three pseudopotential kinds** (`strain_response`): the augmentation charge $Q_{ij}(\mathbf{r})$ is a function of the *cell*, so a homogeneous strain deforms the table it is tabulated on where a displacement only translates it, and on top of that come the four terms the dynamical matrix needed. Against a central difference of *re-converged* strained runs — which needs no `ph.x`, since `ph.x` has no strain perturbation — it agrees to **4.6e-4** (ultrasoft) and **4.7e-4** (PAW) against a norm-conserving 1.9e-4 on the same cell. The *derivatives* below are norm-conserving still — but the variational second-order energy they all stand on is not: it reproduces `dielec.f90`'s dielectric constant to **3.4e-10** (ultrasoft) and **6.9e-11** (PAW) against a norm-conserving 8.4e-10, which took four terms that each vanish when S is the identity. One of them is worth stating on its own, because it is a distinction rather than a term: a *state* is projected with $1-\sum|\psi\rangle\langle\psi|S$ and a *right-hand side* with $1-\sum S|\psi\rangle\langle\psi|$, and collapsing the two — which is exact at $S=1$ — costs a factor of fourteen. The third derivative in this *strain* coordinate is what stays norm-conserving, and how far off it is has been measured rather than left unknown: both of the tangents that closed the Raman tensor transfer, and take it from **4.6e-2** to **1.3e-2** against a central difference over re-converged strained cells, where the norm-conserving control on the same script is **2.3e-4**. What is left is entirely the db term — the same -1.72 of 112 on ultrasoft and on PAW, which is what says it is structural. One candidate for it is excluded by measurement: writing the commutator that sources db with the multiplier matrix rather than the frozen scalar eigenvalue closes this coordinate (1.7e-4) and breaks the *displacement* one, taking the Raman tensor from 1.2e-4 to 1.14e-3. `strain_response` solves the first-order problem for a strain — Abinit's metric-tensor formulation, obtained for nothing here because the cell was already written in reduced coordinates — and two things follow from it:

$$C_{ijkl}=\frac{\partial\sigma_{ij}}{\partial\varepsilon_{kl}}, \qquad \frac{\partial\chi_{ij}}{\partial\varepsilon_{kl}} \ \text{(elasto-optic)}$$

The second is a **third** derivative of the energy, and it comes from one `jvp` of the second-order energy at frozen first-order wavefunctions — the $2n+1$ theorem again. Through the thermodynamic identity of Tanner, Bousquet and Janolin it gives the four electrostriction tensors m, q, M and Q.

```
from defumat.response import elastic_constants, electrostriction

es = electrostriction(calculation, scf)     # solves the strain response itself
print(es.m, es.q, es.M, es.Q)              # the four electrostriction tensors
print(es.photoelastic)                      # the elasto-optic tensor
print(es.elastic.voigt)                     # C_ij in Voigt notation, GPa

# or the elastic constants alone, from a strain response you already have
C = elastic_constants(calculation, scf.wavefunctions, scf.eigenvalues,
                      scf.density, es.strain)
print(C.tensor.shape, C.voigt)
```

Converged silicon gives C_{11} = 198.5 GPa and C_{12} = 68.9, against a measured 165.7 and 63.9; the three independent components of $\partial\epsilon/\partial\varepsilon$ match a central difference over re-converged strained cells to **2e-4**, which is that difference's own floor, and the rank-4 tensor is cubic to 3e-14 with nothing imposing it.

Refuses. Both are norm-conserving, `nspin = 1`, insulators and **clamped-ion**. A diverged first-order solution is refused rather than consumed in silence (`require_converged_responses`) — at QE's `alpha_mix = 0.7` the strain response of a *slab* diverges, and consuming it gave a C_{ijkl} that was not even symmetric under $C_{ijkl} = C_{klij}$. The elastic constants additionally need the whole k-grid rather than a wedge, because their functional builds its own density and symmetrises it as a scalar inside the chain rule.

7.2 The piezoelectric tensor

The polarization a strain induces is the stress an electric field induces — one mixed second derivative read either way round:

$$e_{k,ij} = \frac{\partial P_k}{\partial \varepsilon_{ij}} = \frac{\partial \sigma_{ij}}{\partial \mathcal{E}_k} = -\frac{1}{\Omega}\frac{\partial^2 E}{\partial \varepsilon_{ij}\,\partial \mathcal{E}_k}$$

It is the Born charge's construction with one coordinate changed. Z^* is a `jvp` of the *force* along the field's response; the force is `jax.grad` of the frozen-state energy in the atomic positions and the stress is `jax.grad` of the same functional in a strain, so this is a `jvp` of the *stress* along the same response — three of them, one per field direction, on top of a dielectric constant that was going to be solved anyway. The orthonormality constraint contributes nothing here, because $\langle\psi|\hat{S}|\psi\rangle$ is a sum over a sphere of integers and carries no cell.

```
calc = Calculator.from_file("alas.in", pseudo_dir="pseudo")

piezo = calc.get_piezoelectric_tensor()
print(piezo.voigt)              # (3, 6) in C/m^2, columns xx yy zz yz xz xy
print(piezo.e14)                # -0.7638: zincblende's one component
print(piezo.e.shape)            # (3, 3, 3) in e/bohr^2, e[k, i, j]
print(piezo.dielectric.epsilon) # the field response is shared, not re-solved
```

`pw.x` computes no piezoelectric tensor — the word occurs once in the whole distribution, in a citation in a comment — and Elk's task 380 reaches it by a finite difference of the Berry-phase polarization over one full ground state per strain tensor. So the validation is internal, and there are four independent statements. Silicon is centrosymmetric and its whole tensor vanishes (2.4e-5 C/m^2 against AlAs's 0.764 from the same code, with nothing imposing it on a `nosym` run); AlAs is $\bar{4}3m$ and only $e_{14} = e_{25} = e_{36}$ survive, to 1.7e-14; an eight-point wedge reproduces the sixty-four-point closed grid to **4.5e-9**; and the same mixed derivative contracted the other two ways — `zstar_eu.f90`'s expression with a strain label, and the strain response against the field's bare perturbation — agree to **6.2e-15** and **1.3e-7**. What anchors the sign and the normalisation is that the *same* assembly run in the position coordinate is the Born charge, and that is `ph.x`'s number.

Refuses. Clamped-ion — the atoms are carried by the strain and do not relax, which is also what Elk's task computes; the internal-strain term that makes the constant comparable with experiment is not here, and for zincblende it very nearly cancels this one. And **polar crystals by name**: what the derivative above gives is the *improper* tensor, and the proper piezoelectric response differs from it by $\delta_{ki}P_j - \delta_{ij}P_k$, which needs the spontaneous polarization itself. Those terms vanish identically whenever the two Cartesian labels they pair are different — so e_{14} of a zincblende crystal never carries the ambiguity — and they vanish for every component of a class that admits

no invariant vector, which is what is checked. **Ultrasoft and PAW**, and that one is a gap rather than a missing term: nothing in the assembly is norm-conserving and the displacement leg of it is the Born charge, which is validated on all three kinds, but every ultrasoft and PAW crystal here is centrosymmetric and agrees with zero whatever is wrong. Beyond that: insulators, `nspin = 1`, and everything the Sternheimer regime and the differentiable cell refuse (a metal, a spin spiral, a magnetic field, a shifted grid run with `nosym`).

7.3 Spin-polarized response

The Sternheimer equation is solved per spin channel,

$$\left(H_\sigma - \varepsilon_{n\sigma} S + \alpha_{pv} Q_\sigma\right) |\Delta\psi_{n\sigma}\rangle = -P^{+}_{c,\sigma}\, \Delta V_\sigma\, |\psi_{n\sigma}\rangle,$$

with the occupied manifold $P_{c,\sigma}$ built from *that channel's* filling: $N^{\mathrm{occ}}_{\uparrow} = \mathrm{NINT}(n_{\uparrow})$ and $N^{\mathrm{occ}}_{\downarrow} = \mathrm{NINT}(n_{\downarrow})$, which for a magnetic insulator are different numbers. Quantum ESPRESSO never meets this because LSDA doubles `nks` there and each spin channel is a separate k-point.

```
# chi_0 for a spin-polarized run -- the part that is validated unconditionally.
from defumat.response.sternheimer import make_sternheimer

solver = make_sternheimer(calculation, result, spin_polarized=True)
drho = solver.chi0(dv)          # dv is (2, n1, n2, n3) on the dense grid
```

Validated against a central difference of the density under a spin-dependent probe potential — 1.8×10^{-6} on an antiferromagnetic hydrogen chain (a smeared metal) and 1.1×10^{-6} on triplet O_2 (the sliced branch, ultrasoft) — and against the identity that a cell with no magnetization gives the same ε_∞ at `nspin = 1` and `nspin = 2`: 6.2×10^{-14} on 13.806646105. The probe must differ between the channels, because χ_0 is block diagonal in spin and one equal in both would not tell the blocks apart.

The *screened* response and the Born effective charges are validated for `nspin = 2` as well, on triplet O_2 against the vendored `ph.x`: $\varepsilon_\infty = \mathrm{diag}(1.11091441, 1.11091441, 1.19799977)$ against $(1.110915996, 1.110915996, 1.198004867)$, and Z^* against the *raw* (no acoustic sum rule) block, 0.13372 and 0.20042 against 0.13367 and 0.20023. Two things had to be right for those and neither was a new derivation. The exchange–correlation kernel is *defined* to be zero where the magnetization reaches the charge — `dmxc_lsda` does not regularise $|\zeta| = 1$, it skips it — so what was missing was that convention, applied to the *argument* the derivative is taken at rather than to its value. And the orthonormality multipliers carry a weight on the column of Λ_{mn} and need a *mask* on its row: the solver keeps $\max(N^{\mathrm{occ}}_\sigma)$ bands in every channel because the shape is static, so a magnetic insulator's minority channel carries empty bands whose rows are not removed by the zero weight. That leak is worth a factor of three on Z^*_{xx}, and nothing already validated can see it — the term it feeds is dS/du, which vanishes for a norm-conserving dataset, and the mask is all ones whenever the two channels are filled to the same depth.

Refuses. `tot_magnetization` together with a smearing: the two channels have separate Fermi levels and `Smearing` carries one scalar `ef`. A filling whose boundary lands inside a *degenerate multiplet* — a Hund's-rule atom is exactly this case — where the solve converges and returns an answer 100 % away from a finite difference, because which member falls below the cut is arbitrary; that is the physics, not a missing term.

The dynamical matrix, the strain response, the elastic constants, the Raman tensor and the electrostriction tensors for `nspin = 2`: the solve is spin-polarized, their second-derivative assembly is not — *except* the Born effective charges, which are validated above. The *screened* response of a magnetic system with vacuum: for `nspin = 2` the kernel `dv_of_drho` is the second derivative of the LSDA energy in the two channel densities and diverges wherever a channel density reaches zero — measured on triplet O_2, 1504 of 91125 grid points and exactly 1504 NaN — *lifted for the LDA*, which is the paragraph above, and standing for a **GGA**, whose `dgcxc_spin` carries its own density and ζ gates that nothing here has transcribed. And any potential-only meta-GGA (`tb09`, `bj06`), which has no total energy to differentiate — refused by name now rather than surfacing as `v_of_rho` asking for `tau`.

7.4 Magnons: the transverse spin susceptibility

A magnon is the collective precession of a magnet's own magnetization: a spin wave. It lives at the pole of the transverse susceptibility, which obeys a Dyson equation with *no* Coulomb term — a transverse spin fluctuation moves no charge, and the Hartree interaction belongs to the charge channel:

$$\chi^{+-} = \left[1 - \chi_0^{+-} f_{xc}^{+-}\right]^{-1} \chi_0^{+-}, \qquad f_{xc}^{+-}(\mathbf{r}) = \frac{B_{xc}(\mathbf{r})}{m(\mathbf{r})}.$$

The Kohn–Sham part χ_0^{+-} is a sum over pairs with one member in each spin channel, the majority state at $\mathbf{k}$ and the minority at $\mathbf{k}+\mathbf{q}$; its own spectral weight is the **Stoner continuum** of independent spin flips, which costs at least the exchange splitting. The kernel pulls a mode out from underneath that continuum, and that mode is the magnon.

The kernel is not a second derivative and this is why it works for a GGA. Rotate the whole ground state rigidly by an angle θ: then $\delta\mathbf{m}_\perp = \theta\,\mathbf{m}$ and $\delta\mathbf{B}_{xc,\perp} = \theta\,\mathbf{B}_{xc}$, so the transverse response of the exchange–correlation field to the magnetization is B_{xc}/m — an exact consequence of rotational invariance of $E_{xc}[n,|m|]$, whatever the functional. It also means the Goldstone theorem holds *by construction*, so the residual below tests the assembly and the truncations rather than the physics of f_{xc}.

```
import numpy as np
from defumat import Calculator

calc = Calculator.from_file("ni-fcc-magnon.in")

# one wavevector: the spectral function, and the Stoner continuum under it
chi = calc.get_spin_susceptibility((0.0, 0.0, 0.0), np.linspace(0.0, 0.06, 9),
                                   nbnd=30, ecut_response=60.0)
chi.goldstone              # how far X_0 B_xc = m is from holding: the error bar
chi.enhancement            # lambda_max(X_0 F) at omega = 0; one at q = 0
chi.magnon                 # the pole, in Ry, or None where there is none
chi.magnon_mev
chi.spectral_function      # -Im chi_00 / pi
chi.kohn_sham_spectral_function
chi.method                 # "crossing", "peak" or "unstable"

# a path: one fixed-density run serves every q
disp = calc.get_magnon_dispersion([(0, x, x) for x in (0.0, 0.25, 0.5)],
                                  np.linspace(0.0, 0.06, 9),
                                  nbnd=30, ecut_response=60.0,
```

```
                                goldstone_correction=True)
disp.energies_mev          # (nq,), nan where the grid holds no pole
disp.enhancements          # > 1 is an instability, not a magnon
disp.kernel_scale
```

The functional entry points are `defumat.workflows.run_spin_susceptibility` and `run_magnon_dispersion`.

q must be a difference of two k-points of the run's own grid. That is where the states at $\mathbf{k}+\mathbf{q}$ come from — there is no second diagonalisation — and it is Elk's requirement that `vecql` be commensurate with the mesh, reached by construction. What survives of the $\mathbf{k}+\mathbf{q}$ problem is an umklapp shift of one gather index, and $\chi_0(\mathbf{q}+\mathbf{G}) = \chi_0(\mathbf{q})$ under a relabelling of the matrix's own $\mathbf{G}$ index holds to 1.3×10^{-17}, which is what says that shift is right.

The pole is found as a root, not as a peak. Below the Stoner onset $\lambda_{\max}(\chi_0(\omega) f_{xc})$ is real and rises monotonically, so the magnon is the root of $\lambda_{\max} = 1$ — no broadening enters it, a handful of frequencies suffice, and it reports *below zero* where the state is unstable instead of returning the frequency grid's own edge.

The Goldstone residual is the number to read first. A global spin rotation costs no energy, so $\chi_0(\mathbf{q}{=}0, \omega{=}0)\, B_{xc} = m$ exactly — but only with a complete band set *and* a complete $\mathbf{G}$-set, so what `goldstone` reports is a double truncation. **Which axis binds depends on the element.** On hydrogen the band count does it: $8.0 \to 0.50$ per cent over $n_{\text{bnd}} = 12 \to 140$. On fcc nickel the band count does nothing at all (10.02, 10.13, 10.22 per cent at 30, 60, 100 bands) and only `ecut_response` moves it: $10.0 \to 6.1 \to 2.0$ per cent at 12, 30, 60 Ry, because B_{xc} of a 3d shell has structure a small sphere cannot represent.

Against Elk's committed reference, nickel's magnon agrees to 0.8%. Elk ships an `Ni-magnetic-response` example — fcc nickel at $\mathbf{q} = (0.1, 0.1, 0)$ on a 10^3 shifted grid — whose pole sits at 117.92 meV. Its input applies a small magnetic field, which is worth 1.99 meV of Zeeman gap, so its field-free magnon is 115.93 meV against **115.04** here on the same cell, grid and wavevector — and 123.08 with the Goldstone correction applied, which moves it *away*, so neither is chosen for agreeing. That is across an all-electron LAPW basis versus a norm-conserving plane-wave one, and across a 25 Ry response sphere versus 60. The moment agrees on the same run (0.6178 against 0.6152 μ_B), and so does the sign of the static χ. **That 0.7% of kernel is worth 7% of magnon** is the number to take away: it is how steeply $\lambda(\omega)$ crosses one, and so how far the Goldstone residual has to fall before a magnon energy is worth a second digit.

An enhancement above one is an instability, and that is a result. It says the collinear state is not a minimum at that wavevector — the transverse susceptibility has changed sign — so the ferromagnet is not the ground state there. Nickel's enhancement *falls* away from $\mathbf{q} = 0$, which is what a stable ferromagnet looks like. Most simple models do not: a half-filled band on any lattice prefers antiferromagnetic order, so an fcc hydrogen crystal comes out soft at finite $\mathbf{q}$ at every spacing tried, and the calculation says so rather than producing a plausible dispersion anyway.

Do not compare that against a spiral $E(q)$ without checking what the spiral converged to. It is the obvious cross-check and it does not work: on fcc hydrogen the 90-degree spiral at $q_3 = \frac{1}{2}$ converges to $|m| = 0.0001$, the *nonmagnetic* solution, so its energy gain is demagnetization and not a spin wave; a 15-degree cone collapses the same way, because nothing holds it. Elk's own magnon-spiral example runs `fsmtype = -1` with a large field for exactly that reason.

`goldstone_correction=True` scales the kernel by $1/\lambda_{\max}(\mathbf{q}{=}0)$, the same factor at every $\mathbf{q}$, so the uniform mode sits at exactly zero. It is what the magnon literature does, it is off

by default, and it is honest only if the residual error is **q**-independent — check that by running the dispersion at two (n_{bnd}, `ecut_response`) settings.

Refuses. `nspin != 2`: without a collinear magnetization the transverse block does not decouple and B_{xc}/m does not exist, and a **noncollinear** ground state needs the full 4×4 spin-density response, which is a different object. **Ultrasoft and PAW**, because $\langle u_i|e^{-i(\mathbf{q}+\mathbf{G})\cdot\mathbf{r}}|u_j\rangle$ is not the plane-wave overlap once the charge is not all in $|\psi|^2$. A **spin spiral**, whose rotating frame is not the one the kernel was derived in. A **Hubbard** U, which responds to the magnetization too. An **applied magnetic field or a constrained moment**, because the rotation argument assumes the energy is invariant under a global spin rotation and a field is exactly what breaks it — a magnon in a field is gapped by the Zeeman energy, which this construction does not carry. A **potential-only meta-GGA**, there being no E_{xc} for that argument to be about. And a **symmetry-reduced k-set**: $\chi_0(\mathbf{G},\mathbf{G}')$ is a matrix in two **G** indices that nothing here symmetrises, and the $\mathbf{k}+\mathbf{q}$ fold needs the grid closed under translation by **q** in any case.

8 Optical spectra and excitons: TDDFT

The response above is a *static* one, solved orbital by orbital. An absorption spectrum needs the frequency axis and needs the susceptibility as a **matrix** in reciprocal lattice vectors, so this is the one place where a sum over states earns its keep. `run_absorption` builds

$$\chi^0_{\mathbf{G}\mathbf{G}'}(\omega)=\frac{1}{\Omega}\sum_{\mathbf{k}}\sum_{ij}\frac{(f_i-f_j)\,\rho^{ij}_{\mathbf{G}}\,\rho^{ij\,*}_{\mathbf{G}'}}{\epsilon_i-\epsilon_j+\omega+i\eta},\qquad \rho^{ij}_{\mathbf{G}}=\langle u_i|e^{-i\mathbf{G}\cdot\mathbf{r}}|u_j\rangle$$

and solves the Dyson equation with an exchange-correlation kernel:

$$\epsilon^{-1}=1+X\big(1-X-\tilde f_{\mathrm{xc}}X\big)^{-1},\qquad X=v^{1/2}\chi^0 v^{1/2},\qquad \tilde f_{\mathrm{xc}}=v^{-1/2}f_{\mathrm{xc}}v^{-1/2}$$

The **bootstrap** kernel of Sharma, Dewhurst, Sanna and Gross (*PRL* **107**, 186401 (2011); Elk's `fxctype = 210`) is that equation's other half, so the two are solved together to self-consistency:

$$f^{\mathrm{BS}}_{\mathrm{xc}}=-\frac{\epsilon^{-1}(\mathbf{q},0)\,v(\mathbf{q})}{\epsilon_0(\mathbf{q},0)-1},\qquad \epsilon_0=1-v\chi^0$$

It is parameter-free and diverges as $1/q^2$, which is what binds an electron–hole pair; ALDA's kernel is finite at $\mathbf{q}=0$ where v diverges, so its head and wings vanish *identically* and no amount of it can bind one.

Quantum ESPRESSO has no counterpart. `TDDFPT/` is a Liouville–Lanczos solver with RPA and ALDA; it has no bootstrap kernel and no Dyson equation in **G** space. The reference is Elk.

```python
from defumat.workflows import run_absorption
import numpy as np

omega = np.arange(0.0, 0.6, 0.005)          # Ry
spectrum = run_absorption(system, pseudos, scf.density, omega,
                          kernel="bootstrap",   # or rpa / alda / lrc / bootstrap-1
                          nbnd=60, ecut_response=8.0,
                          broadening=0.012, scissor=0.05)

print(spectrum.absorption)                  # Im eps_M, isotropic average
```

```python
print(spectrum.frequencies_ev)              # the same grid in eV
print(spectrum.alpha, spectrum.iterations)  # the LRC-equivalent head, and the
                                            # bootstrap's fixed-point count
print(spectrum.static_residual)             # how far the band truncation reaches
```

`chi_0` is the whole cost and the kernel is nearly free, so comparing kernels means building it once: `independent_response` returns it and `solve_dyson` screens it, which is what `run_absorption` does internally and what the notebook shows.

Three knobs here are this code's own. `ecut_response` (Elk's `gmaxrf`) selects the **G**-set the matrix is built over and has its own convergence: too small and the local-field effect is missing, and the cost is quadratic in it. `scissor` is *part of the method* rather than a convenience — the paper's Eq. (3) replaces χ^0 by a gap-corrected model response, and the velocity matrix elements are renormalised by $e_{ij}/(e_{ij}\mp\Delta)$ along with it. And `static_residual` is the diagnostic for the one error this phase cannot refuse: how many empty states the sum ran over. It is $\epsilon_M(0)$ from this run, in RPA, minus the same quantity from the Sternheimer solve, which is band-*complete*; it is kernel-matched on purpose, since differencing a bootstrap number against a Sternheimer one that contains f_{xc} would measure the kernel instead.

A trap that is invisible in the answer: the macroscopic dielectric function is the inverse of the 3×3 *head* of ϵ^{-1}, not the head of the inverse of the whole matrix. The second is the microscopic ϵ, whose head in RPA is exactly the no-local-field result; it is smooth, positive, has the right peaks and is 9% too large on silicon. Both are returned (`epsilon` and `epsilon_no_local_fields`), because the gap between them *is* the local-field effect.

Validated by an identity rather than against another code's spectrum: the same $\epsilon_M(0)$ is reached by this sum over states plus a Dyson inversion and by the projected conjugate-gradient solve of `dielectric_tensor`, which shares no machinery with it and never sees an empty state. On silicon the two agree to **1.3e-2** on a constant of 22 — and that residue is the band truncation, which is why it is reported rather than tuned away. The `screening = "hartree"` switch on `dielectric_tensor` exists for this comparison: the identity holds only when both sides use the same kernel.

Refuses. Finite **q** (the perturbed states would live at $\mathbf{k}+\mathbf{q}$); ultrasoft and PAW (every matrix element gains the augmentation charge $Q_{ij}(\mathbf{G})$); metals (the f_i-f_j weight kills the $i=j$ term, so the intraband Drude response is absent entirely — Elk's `tddftlr` does not add it either); `nspin` $\neq 1$, noncollinear magnetism, spin–orbit coupling, spin spirals and a Hubbard U; a symmetry-reduced k-set, because symmetrising $\chi^0_{\mathbf{GG}'}$ rotates *two* **G** indices at once and the rank-n symmetriser is Cartesian; the `alda` kernel with a gradient-corrected functional, since ALDA's kernel is pointwise in the density and a GGA's is not; and **non-convergence of the bootstrap fixed point is an error**, as it is in `tddftlr.f90`, because a spectrum from a kernel still in motion is the spectrum of nothing.

8.1 The conductivity tensor, the Kerr angle and the anomalous Hall effect

The whole complex tensor, interband plus a Drude term:

$$\sigma_{ij}(\omega)=\frac{i}{\Omega}\sum_{\mathbf{k}}\sum_{n,m}\frac{W_n\,(1-f_m)}{\epsilon_{mn}}\left[\frac{v^i_{nm}v^j_{mn}}{\omega-\epsilon_{mn}+i\eta}+\frac{\overline{v^i_{nm}v^j_{mn}}}{\omega+\epsilon_{mn}+i\eta}\right]+\frac{\omega^2_{p,ij}}{4\pi(\gamma-i\omega)}$$

with $v = \partial H/\partial \mathbf{k}$ from one `jvp` of $H(\mathbf{k})$ at a frozen sphere. *That* is the one thing here not transcribed from `dielectric.f90`, and it is not a refinement: `epsilon.x` accumulates $\langle\psi_1|\mathbf{G}|\psi_2\rangle$, and $[H, \mathbf{r}] \neq \mathbf{p}$ when the pseudopotential is nonlocal.

What is new beside `run_absorption` is the **antisymmetric** part, σ_{xy} — the response that rotates light reflected off a magnet, and whose static limit is the intrinsic anomalous Hall conductivity. Time reversal forces it to zero, so a nonmagnetic crystal has none however strong its spin–orbit coupling; a *collinear or coplanar* magnet without spin–orbit coupling has none either, because a spin rotation composed with conjugation survives as an antiunitary symmetry. (A *noncoplanar* magnet does have one with no spin–orbit coupling at all — the topological Hall effect — so the usual pairing is a route and not a theorem.)

```
calc = Calculator.from_file("ni-soc.in", pseudo_dir="pseudo")

sigma = calc.get_optical_conductivity(nbnd=36, window=0.6, nw=200)
print(sigma.sigma_s_per_cm.shape)   # (nw, 3, 3) in S/cm
print(sigma.kerr)                   # complex Kerr angle, degrees, per frequency
print(sigma.hall_conductivity)      # the static antisymmetric part, S/cm
print(sigma.plasma_ev)              # hbar omega_p, eV, a (3, 3) tensor
print(sigma.dielectric)             # 1 + 4 pi i sigma / omega
print(sigma.fsum, sigma.band_cut_gap)   # two of the three diagnostics
print(sigma.degenerate_pairs)       # the third: pairs the guard dropped, usually 0
```

There is no reference output for any of it, so the validation is internal and analytic. The *symmetric* part reproduces the independent-particle dielectric function of the TDDFT stack to **3.6e-14**, which is a different assembly (a Dyson solve over a response sphere) reaching `ph.x` through `dielectric_tensor`. The **f-sum rule** is the absolute anchor: $\int_0^\infty \mathrm{Re}\,\sigma_{aa}\,d\omega$ must equal $(\pi/2\Omega)\sum W_n\langle n|\partial^2 H/\partial k_a^2|n\rangle$, and the familiar $\pi n_e/2$ is only the *local* Hamiltonian's special case — silicon's diamagnetic weight is measurably **0.943** and not one. Measured against a central difference of the velocity operator, the ratio is 1.258 on a 4^3 grid, 1.045 on 6^3 and **0.990** on 8^3: it converges in the *k-grid*, because what separates the two is $\sum_k w_k\,\partial^2\epsilon_n/\partial k^2$, which vanishes over the zone by periodicity and not on a coarse mesh. Aluminium's plasma frequency comes out at **12.98 eV** against a free-electron 16.27, which is the zone-boundary gaps removing Fermi surface, and its tensor is isotropic to **1.7e-4** eV with nothing imposing that.

Refuses. Ultrasoft and PAW, for the Kubo curvature's reason: with a moving $\hat{S}$ the current operator carries $\epsilon_n\,\partial S/\partial k$ off the diagonal, and that term is identically zero for a norm-conserving dataset, so nothing validated here can see whether its convention is right. A **spin spiral**, whose two spinor components live on different spheres. A **symmetry-reduced k-set**, because the antisymmetric part is an axial vector — the escape is the whole unshifted grid, which is closed under the point group. And **`nspin` = 2**, whose two channels are two band structures whose conductivities add; a spinor run is the regime a magneto-optical spectrum wants anyway, since σ_{xy} needs spin–orbit coupling and collinear spin has none.

Three things are reported rather than refused, and all three must be read. `band_cut_gap` is the gap between the last band summed and the first dropped: stopping *inside* a degenerate multiplet keeps some members and drops others, and silicon's antisymmetric σ — zero by time reversal — comes out at 4.0e-13 when the cut falls at a real gap and at 1.0e-5 when it does not. `degenerate_pairs` counts the occupied/empty pairs that came back closer together than `degeneracy_tol` (1e-5 Ry, what an SCF con-

verges an *empty* band to) and were therefore dropped. Zero is the ordinary case. When it is not zero it usually still does not matter — on the smeared frequency route the two orderings of such a pair cancel analytically, because what survives the $1/\epsilon_{mn}$ is $w_k[f(\epsilon_n)-f(\epsilon_m)]$ and that is itself linear in the gap — and the code warns exactly where that cancellation fails: `method = "curvature"`, whose $1/\epsilon_{mn}^2$ leaves a $1/g$ divergence, and a fixed-occupation run, where f is not a function of energy at all. And the **anomalous Hall conductivity of a metal is mesh-hungry**: the static Berry-curvature integral is concentrated on near-degeneracies at the Fermi surface, so it is quoted with its k-convergence and not on its own.

8.2 The shift current: the bulk photovoltaic effect

Illuminate a crystal with no inversion centre and it carries a direct current, with no junction and no built-in field. The intrinsic part of that is the *shift current*: the photoexcited electron is born displaced, and what the current counts is the real-space shift between the valence and conduction Wannier centres. It is a second-order optical response, $J^a = 2\sigma^{abc}(0;\omega,-\omega)E_b(\omega)E_c(-\omega)$, with

$$\sigma^{abc}(\omega) = -\frac{i\pi e^3}{4\hbar^2}\int [d\mathbf{k}] \sum_{n,m} f_{nm}\left(I^{abc}_{mn} + I^{acb}_{mn}\right)\left[\delta(\omega_{mn}-\omega)+\delta(\omega_{nm}-\omega)\right], \qquad I^{abc}_{mn} = r^b_{mn}\, r^{c;a}_{nm}.$$

The whole difficulty is $r^{c;a}_{nm}$, the *generalised derivative* $\partial_a r^c_{nm} - i(A^a_{nn}-A^a_{mm})r^c_{nm}$ — neither piece of which is separately computable, since the diagonal Berry connection is gauge dependent and is not a function of H at one $\mathbf{k}$. The escape is the Aversa–Sipe sum rule, which writes the covariant object entirely in gauge-free quantities living at a single $\mathbf{k}$:

$$r^{c;a}_{nm} = \frac{i}{\omega_{nm}}\left[\frac{v^c_{nm}\Delta^a_{nm} + v^a_{nm}\Delta^c_{nm}}{\omega_{nm}} - w^{ca}_{nm} + \sum_{p\neq n,m}\left(\frac{v^c_{np}v^a_{pm}}{\omega_{pm}} - \frac{v^a_{np}v^c_{pm}}{\omega_{np}}\right)\right],$$

with $v^a_{nm} = \langle n|\partial_a H|m\rangle$ and $w^{ab}_{nm} = \langle n|\partial_a\partial_b H|m\rangle$. Both are derivatives of code that already exists: v is one `jvp` of $H(\mathbf{k})$ at a frozen sphere and w is that `jvp` differentiated by a second one. Nothing is transcribed. The interband dipole $r^a_{nm} = -i\,v^a_{nm}/\omega_{nm}$ is *exact* for an eigenstate of the full $H(\mathbf{k})$, which is why a plane-wave code needs no counterpart to Wannier90's position-operator matrices.

```
calc = Calculator.from_file("alas-raman.in", pseudo_dir="pseudo")

sc = calc.get_shift_current(nbnd=14, window=0.9, nw=180)
print(sc.sigma.shape)            # (nw, 3, 3, 3) in A/V^2, real
print(sc.component(0, 1, 2))     # sigma^xyz(omega), -43m's only free component
print(sc.frequencies_ev)         # the photon energy axis in eV
print(sc.truncation)             # the band-truncation diagnostic: read it
print(sc.band_cut_gap)           # where the band set was cut, in Ry
```

There is no reference binary — `pw.x` computes no photocurrent of any kind, and Elk's `nonlinopt.f90` is second-harmonic generation, a different response — so the validation is internal and it is three statements. The **sum rule against a parallel-transport finite difference** of the dipole, which shares nothing with it: no second derivative, no intermediate sum, only the phases of neighbouring $\mathbf{k}$-points fixed to one another. That comparison is decisive in exactly one form, because the sum rule is an *identity over a complete basis*: held on a frozen sphere of 158 plane waves, AlAs agrees to **1.8e-4** at 158 bands against

4.3e-2 at 120 and 6.0e-2 at 20. A residue that falls off a cliff at completeness is a correct assembly with a truncated sum; one that plateaus is a bug. Then the **crystal class**: AlAs comes out exactly $\bar{4}3m$ on a `nosym` grid, only the components with all three labels different surviving, while centrosymmetric silicon gives zero from the same machinery. And the **absorption edge**: σ^{xyz} is 6.9e-25 A/V^2 at 2 eV against a peak of 1.1e-4 near 4.4 eV, which is the one check no index bookkeeping can pass by accident — no absorption, no photocurrent.

All three are blind to an overall constant, and this document says so rather than leaving it to be discovered: a σ wrong by a factor of two is still exactly $\bar{4}3m$, still zero on silicon and still zero below the gap. Two further checks close that gap. The **spin sum** — the same cell run `nspin` = 1 and as a two-component spinor gives the same tensor to **4.6e-9**, peak ratio 1.000000. And the **overall scale against the published literature**, where the *ordering* is what makes the comparison sharp rather than merely consistent. AlAs's first peak converges to **35.0** μA/V^2 at 4.17 eV (33.9 at `ecutwfc` = 16, then 33.7, 34.8 and 35.0 at 22 with 14, 22 and 30 bands), where calculations across the fourteen III–V and II–VI zincblende semiconductors span 14 μA/V^2 (CdSe, smallest) to 83 (AlSb, largest) and find the aluminium compounds the strongest of the family. A factor of two either way breaks that ordering rather than moving the number: 17 would put AlAs at the bottom of the family, below the cadmium chalcogenides, and 70 would put it beside AlSb, the heaviest and narrowest-gap member. The number to compare is the peak below about 6 eV, which is the range published spectra cover. That sweep also showed `truncation` to be *predictive* rather than merely indicative: it falls 3.5% to 1.0% over the band sweep while the value moves 3.9%.

The convention is declared, because the literature has two. What is implemented is $j^a = 2\sigma^{abc}(0;\omega,-\omega)\,\mathrm{Re}[\mathcal{E}_b(\omega)\mathcal{E}_c(-\omega)]$, the definition of the paper the formulae above come from. Some authors write $J = \sigma\mathcal{E}\mathcal{E}$ and report numbers twice as large for the same physics, so a comparison against a paper that does not state its normalisation is worth nothing to better than a factor of two.

Refuses. Ultrasoft and PAW, for the Kubo curvature's reason one order further out: with a moving $\hat{S}$ the dipole carries $\partial S/\partial k$ and its derivative carries $\partial^2 S/\partial k_a \partial k_b$, both identically zero for a norm-conserving dataset, so nothing validated here can see whether their convention is right. A **spin spiral**. A **symmetry-reduced** k**-set**, because σ^{abc} is a polar rank-3 tensor and a wedge sum is not the cell's until it is averaged over the point group; the escape is the whole unshifted grid. A **metal**, whose partially filled bands contribute pairs with vanishing ω_{nm} where every denominator above diverges — the Fermi-surface terms that replace them are a different assembly. **DFT+**U, whose Hubbard term is built from $\phi_U(\mathbf{k}+\mathbf{G})$ and so carries a velocity and a second velocity of its own. And **`nspin` = 2**. A *spinor* run is supported and tested.
The band count is reported rather than refused, and it is the largest approximation here. The intermediate sum over p converges only as it approaches completeness, so a production band count carries a few per cent that no symmetry check sees — `truncation` is the in-run estimate and is taken over the top *quarter* of the bands, because dropping one band reports 5.3e-5 where the error is 4 per cent. The route that would remove it is written down in `NONLINEAR.md`: the two sums are a Sternheimer solve rather than a spectral sum, and what stops it being taken is that the operator to invert is indefinite. `band_cut_gap` carries the same warning it does for the conductivity.

8.3 Second-harmonic generation

Shine light of frequency ω on a crystal with no inversion centre and some of it comes back out at 2ω. The tensor that says how much is the second-order optical susceptibility $P^a(2\omega) = \chi^{abc}(-2\omega;\omega,\omega)\,E_b(\omega)E_c(\omega)$, a polar rank-3 tensor, symmetric in its last two labels and identically zero in every centrosymmetric crystal. It is a sum of three physically distinct pieces — the pure interband term χ_{II}, the intraband modulation of it η_{II}, and the modulation of the transition energy $\frac{i}{2\omega}\sigma_{II}$ — each of which is a contraction of the interband dipoles $r^a_{nm} = -i\,v^a_{nm}/\omega_{nm}$ over two resonant denominators, one at ω and one at 2ω:

$$\chi^{abc}(-2\omega;\omega,\omega) = \frac{1}{\Omega}\sum_{\mathbf{k},m,n}\left[\frac{c^{(1)}_{mn}}{\omega_{mn}-\omega+i\eta} + \frac{c^{(2)}_{mn}}{\omega_{mn}-2(\omega-i\eta)}\right],$$

where the coefficients $c^{(1,2)}_{mn}$ carry a further sum over an intermediate state and are built once, before the frequency axis is touched. This is the same sum over states the shift current uses, contracted differently; it needs no second derivative of H at all, because that triple sum *is* the sum-rule expansion of the generalised derivative written out rather than collapsed.

```
calc = Calculator.from_file("alas-shg.in", pseudo_dir="pseudo")

shg = calc.get_shg(nbnd=22, window=0.6, nw=240, broadening=0.010)
print(shg.chi.shape)                  # (240, 3, 3, 3), complex, in pm/V
print(shg.component(0, 1, 2)[0])       # chi^xyz at the lowest frequency
print(shg.d_coefficient(0, 1, 2)[0])   # d = chi/2, what the literature quotes
print(shg.frequencies_ev[:3])          # the FUNDAMENTAL photon energy, in eV
print(shg.truncation)                  # the band-truncation diagnostic
print(shg.chi_ii, shg.eta_ii, shg.sigma_ii)   # the three parts, kept apart
```

There is a real reference here, which is unusual in this corner of the package: Elk's `nonlinopt.f90` (task 125) computes the same tensor, and `tests/data/elk/` carries its output for the same AlAs crystal, the three parts separately as well as their sum. `pw.x` has none — its `el_opt.f90` is the *static* electro-optic response, and the branch that reaches even that is the one this document's Raman section records as broken in the vendored 7.5.

What the comparison is *not* is digit-for-digit, and saying so is part of quoting it: an all-electron LAPW basis is not a norm-conserving pseudopotential one, and a second-order susceptibility carries two energy denominators, so a gap difference is not a scale factor. On the same crystal and the same 6^3 mesh: the **resonance position** agrees to **0.5%** (2.152 eV against 2.163), the **peak height** to **7%** (27.52 a.u. against 25.70), and the **static value** to **11%** (-3.10 a.u. against -3.50). The last is the one that moves with the basis, and it is shown converged: `ecutwfc` 10, 30 and 45 give -2.71, -3.10 and -3.13, so the residue is the pseudopotential rather than the cutoff.

Four internal checks close what the reference cannot. $\bar{4}3m$: AlAs comes out exactly zincblende on a `nosym` grid with nothing imposing it. **Silicon**: an inversion centre forbids every component, and the centrosymmetric control runs through the same machinery. **The two-photon edge**: $\mathrm{Im}\,\chi$ turns on at *half* the smallest direct gap, not at the gap, because the second channel's denominator is $\omega_{mn} - 2\omega$ — the only check here blind to both the scale and the symmetry, and a lost factor of two there moves the absorption edge by an octave. And **the spinor**, which is the scale's own anchor: the same cell run `nspin` = 1 and as a two-component spinor must give the same tensor, which is what catches a factor of two in the spin sum.

The convention is declared, because the literature has two. What is computed is $\chi^{(2)}$, defined by $P^a = \chi^{abc} E_b E_c$; nonlinear-optics papers more often quote $d^{abc} = \chi^{abc}/2$, so a number compared against a published d without halving it is wrong by exactly two and nothing about its shape, symmetry or zeros would say so. `d_coefficient` returns the halved form.

The scissors shift is the one knob worth explaining. An LDA or GGA gap error does not scale a second-order susceptibility, it moves its resonances, and `scissor` applies the usual rigid shift to the empty states. The dipoles are built from the *unshifted* gaps — a scissors correction is a statement about energies, not about wavefunctions — which is what Elk reaches by scaling its momentum matrix elements instead. Validated against Elk's own scissored run: at $\Delta = 0.05$ Ha the 2ω peak moves **0.0502** Ry against a half-scissor of 0.0500, and its height falls to **0.60** of the unscissored value against Elk's 0.58.

Refuses. The same five the shift current refuses, since this is the same velocity matrix elements contracted a different way and it inherits every reason: **ultrasoft and PAW** (a moving $\hat{S}$ puts $\partial S/\partial k$ in the dipole), a **spin spiral**, a **symmetry-reduced k-set** (a polar rank-3 tensor is not the cell's until averaged over the point group; the escape is the whole unshifted grid), a **metal**, **DFT+U**, and **`nspin` = 2**. A *spinor* run is supported and tested.
This does not lift the `chi^(2)` refusal in the Raman section, and the distinction matters. That one is about the $2n+1$ route through the Sternheimer solver, where a field enters only through a source term and the $\langle u_i | r_k | u_j \rangle$ piece has nothing to build it from; a sum over states never needed that object. The electro-optic tensor and a truncation-free static $\chi^{(2)}$ are still refused for the same missing term.
The band count is reported rather than refused, exactly as for the shift current and for the same reason: the sum over the intermediate state is an identity only over a complete basis. `truncation` is the in-run estimate, taken over the top quarter of the bands.

9 Topological invariants

Everything is built from one primitive — the overlap of occupied manifolds at neighbouring k-points,

$$M^{(\mathbf{b})}_{mn} = \langle u_{m\mathbf{k}} | \hat{S} | u_{n,\mathbf{k}+\mathbf{b}} \rangle$$

because a determinant of overlaps is blind to the unitary mixing a degenerate eigensolver leaves. The Berry curvature is the phase of a plaquette of them and the Chern number their lattice sum, which is an **exact integer on any mesh**:

$$F_{12}(\mathbf{k}) = \ln\left[U_1(\mathbf{k}) U_2(\mathbf{k}+\hat{1}) U_1(\mathbf{k}+\hat{2})^{-1} U_2(\mathbf{k})^{-1}\right], \qquad C = \frac{1}{2\pi i} \sum_{\mathbf{k}} F_{12}(\mathbf{k})$$

Z_2 has two independent routes — the flow of Wannier charge centres, and, when there is an inversion centre, the parity product over the eight TRIM points:

$$(-1)^{\nu} = \prod_{i=1}^{8} \prod_{n\ \mathrm{occ}}^{\mathrm{Kramers}} \xi_{2n}(\Gamma_i)$$

```
from defumat.workflows.topology import (run_z2, run_z2_3d,
                                        run_berry_curvature)
```

```
z2 = run_z2(system, pseudos, scf.density, axis=2)   # Wilson loop by default
print(z2.z2, z2.gap_step)   # read gap_step before believing the integer

chern = run_berry_curvature(system, pseudos, scf.density, shape=(12, 12))
print(chern)                # an exact integer on any mesh

nu = run_z2_3d(system, pseudos, scf.density)   # the four 3D indices
print(nu)                   # (nu0; nu1 nu2 nu3)
```

Refuses. A collinear spin-polarized run (two Hamiltonians, so two independent sets of invariants and no statement of which is meant); a Z_2 without spin–orbit coupling, where every invariant is zero for a reason that has nothing to do with the band structure; the parity route without an inversion centre; and a manifold that is not separated from the band above it (`gap_tol`). A PAW run needs the converged `becsum` passed beside the density — `run_z2(..., becsum = scf.becsum)`, which `Calculator` does for you — because `ddd_paw` is a property of the wavefunctions and cannot be rebuilt from a density; leaving it out is worth 1.03 eV in the eigenvalues, and an invariant computed from those is still an integer.

Running both Z_2 routes is the check. Where they disagree the parity one is the answer, because it has no mesh and the Wilson route does. **Two things bite in a plane-wave code and both are silent:** neighbouring k-points do not share a G-sphere, so coefficients are aligned by Miller index; and the wrap at the zone edge is a *shift* of that index, without which the Chern number comes out smooth and non-integer.

9.1 The curvature as a map: the Kubo route

Two constructions of the same quantity, chosen by name:

$$\Omega_n^{12}(\mathbf{k}) = -2\,\mathrm{Im}\sum_{m\neq n}\frac{A^1_{nm}A^2_{mn}}{(\varepsilon_n-\varepsilon_m)^2}, \qquad A^a_{nm} = \langle\psi_n|\,\partial_{k_a}H - \varepsilon_n\,\partial_{k_a}S\,|\psi_m\rangle$$

with `method="kubo"`, against the lattice field strength of Fukui, Hatsugai and Suzuki with `method="fhs"` (the default). The velocity operator is one `jvp` of $H(\mathbf{k})$ at a frozen plane-wave sphere — no dense $H(\mathbf{k})$ is ever formed — and the tangent for a crystal direction d is the reciprocal lattice vector $\mathbf{b}_d$.

```
from defumat.workflows.topology import run_berry_curvature

curvature = run_berry_curvature(
    system, pseudos, density, shape=(4, 4), nocc=4, nbnd=12, method="kubo"
)
curvature.curvature          # (4, 4), Omega(k)
curvature.curvature_by_band  # (4, 4, nocc) -- see the refusal box
curvature.truncation         # what the highest empty band was worth
```

Validated against the FHS flux, which shares no machinery with it: a centred plaquette shrunk around one k-point of zincblende AlAs converges onto the pointwise Kubo value to 3.2×10^{-3}, and the whole 24×24 mesh agrees plaquette by plaquette to 1.45×10^{-4}, improving 65-fold over a sixfold refinement. The velocity matrix elements themselves

reproduce a central finite difference of $H(\mathbf{k})$ to 1.8×10^{-9}, and on centrosymmetric silicon the curvature vanishes *pointwise* to 3.5×10^{-5}.

Refuses. `method="kubo"` is for the map, never for the integer: its denominator $(\varepsilon_n-\varepsilon_m)^{-2}$ is singular at a degeneracy and its Brillouin-zone sum is an ordinary Riemann sum, which converges to an integer without ever being one — `method="fhs"` is the default and is exact on any mesh. **Ultrasoft and PAW are refused by name**: the $\varepsilon_n\partial_k S$ term is identically zero for a norm-conserving dataset, so no validation here can see whether its off-diagonal convention is right, and $q_{ij}(\mathbf{b})$ is the second missing piece; `method="fhs"` carries both correctly on all three dataset kinds. The sum over empty states is truncated at `nbnd` and the truncation is *reported* rather than hidden — read `BerryCurvature.truncation`, or `truncation_abs` wherever symmetry forces the curvature to vanish, since the ratio is then noise over noise. `curvature_by_band` is gauge invariant only for a non-degenerate band; inside a degenerate multiplet only the sum over the members is defined, and `curvature` (the manifold total) always is.
A mesh point where an occupied and an empty band actually *touch* — a Dirac cone, a Weyl node, the closing a topological transition runs through — is a genuine singularity and is **dropped, with a warning naming how many**. `BerryCurvature.singular_points` carries the count, and a nonzero one means the mesh sum is not a Chern number of that state set: the curvature diverges there and a zero was put in its place, which is smooth and plausible and wrong. Move the mesh off the point, or use `method="fhs"`, which needs no gap in a denominator. A band pair counts as touching below 10^{-8} of the band width *over the whole mesh* — a fraction rather than an energy, since a model Hamiltonian carries no unit, and over the whole mesh because for a two-band model the spectrum at one $\mathbf{k}$ *is* the gap.

9.2 The Berry-phase polarization

The polarization of a crystal is not the dipole of its unit cell — that integral depends on where the cell is cut. What is a property of the crystal is a *phase*: the Berry phase the occupied manifold accumulates as $\mathbf{k}$ crosses the zone once along a reciprocal lattice vector $\mathbf{b}_i$ (King-Smith and Vanderbilt, PRB **47**, 1651 (1993)),

$$\Omega\,\mathbf{P} = \sum_i \frac{\varphi_i}{2\pi}\,\mathbf{a}_i, \qquad \varphi_i = \underbrace{\operatorname{Im}\ln\prod_j \det M^{(\mathbf{b}_i/N)}(\mathbf{k}_j)}_{\text{electronic}} + \underbrace{2\pi\sum_a Z_a^{\mathrm{v}}\,s_{ai}}_{\text{ionic}}$$

with M the same overlap the invariants above are built from and s_{ai} the atom's crystal coordinate. It is the *determinant* of the overlap that is kept here where a Wilson loop keeps the eigenvalues, which is the whole difference between the two calculations.

The value is defined modulo a quantum and the result object carries it. A single number is not a physical statement: what is meaningful is a *difference* between two geometries taken on the same branch, which is what a Born effective charge and a piezoelectric constant are made of.

```
from defumat.workflows.polarization import run_polarization

p = run_polarization(system, pseudos, scf.density,
                     gdir=2,            # 0-based; pw.x's gdir = 3
                     nppstr=6,          # as pw.x counts it
                     transverse=(2, 2))
print(p.total_phase, p.quantum)         # -0.2412383  (mod 1)
```

```
print(p.polarization_si)                # -0.34909 C/m^2, mod 1.44708
print(p.strings.phases)                 # one phase per string, four here
```

`Calculator.get_polarization()` reads `lberry`, `gdir` and `nppstr` straight off a `pw.x` input, so a polarization run transfers unchanged. `nppstr` counts the repeated endpoint the way `kp_strings` does; the mesh built here carries `nppstr - 1` distinct points and closes the string with the reciprocal lattice vector, which is the same links and one diagonalisation fewer per string.

Refuses. A **metal** — a Berry phase is a property of a gapped manifold, and a smeared occupation does not say which bands the string carries. **`nspin = 2`**: two channels are two independent string sets and one phase each, which is a layout this does not have rather than a missing term. And a **spin spiral**, whose two spinor components live on spheres centred at $\mathbf{k} \pm \mathbf{q}/2$, so a string overlap is not a single gather. A *spinor* run (`noncolin`) is supported and has its own `pw.x` reference; its quantum is half the scalar one, because a spinor band holds one electron. Ultrasoft and PAW are *not* refused, and both are validated: zincblende SiC matches `pw.x` string by string with an electronic phase of -0.009696 against -0.00970. It had to be SiC rather than silicon, because a centrosymmetric crystal's electronic phase is zero whatever $q_{ij}(\mathbf{b})$ does and so agrees with zero however wrong the augmentation is.

Validated three ways, two of which need no other code. Against `pw.x`'s own `lberry` run on AlAs it agrees *string by string* and on every digit printed (electronic phase 0.00876, total -0.24124). The Zak phase of the SSH model is pinned to 0 or π by symmetry and comes out exactly that on a mesh of *any* size (10^{-10}), and silicon is pinned by inversion to 0 or half a quantum, landing on the half (10^{-8}). Silicon is also the second `pw.x` reference, and the one that exercises the quantum of 2: both its species carry an even valence, where AlAs's arsenic does not. The strongest is $\mathrm{d}P/\mathrm{d}u$ against the Born charges, which come from a Sternheimer solve and share no machinery with a string of overlaps: AlAs on an $8 \times 8 \times 8$ ground state gives $Z^*(\mathrm{Al}) = +2.148$ against `ph.x`'s $+2.142$, charge-neutral to 5×10^{-5}.

9.3 The magnetoelectric tensor

How much electric polarization a magnetic field induces,

$$\alpha_{ij} = \frac{\partial P_i}{\partial B_j},$$

computed Elk's way (task 390): a full ground state at $B \mp \delta/2$ for each Cartesian direction, the Berry-phase polarization of each, and a central difference. The cheap route — one derivative of the magnetization along an electric field's response — needs a noncollinear Sternheimer solve, and an electric field as a ground-state perturbation, which this package does not have.

Two symmetries must both be broken or the answer is zero for a reason that has nothing to do with the calculation: time reversal and inversion each map α to $-\alpha$. Elk's own example is built around the first, and so is the case here — a non-magnetic semiconductor has no linear tensor at all, so a *large* external field is applied to break time reversal by hand and the derivative taken against a small further change in it. The magnetism is induced rather than spontaneous, which is what makes a plain gapped semiconductor usable.

```python
from defumat import Calculator

alas = Calculator.from_file("alas-magnetoelectric.in", pseudo_dir)
me = alas.get_magnetoelectric_tensor(delta=0.02, directions=(2,))
print(me.magnitude)        # 4.4376e-06
print(me.largest_step)     # how close a field step came to a branch
```

Refuses. An unconverged ground state, which is what makes the next limitation visible rather than silent. **Only the column parallel to the seeded magnetization is reachable** on the committed cell: a field along the direction `starting_magnetization` points converges in 12 iterations, and one with a transverse component does not converge in 150, because rotating the moment is the slow mode. It is not the tilt — a pure transverse field fails on its own. Seed the magnetization along the column you want. A run carrying no magnetization vector ($n_{\mathrm{spin,mag}} \neq 4$): the Zeeman term then multiplies nothing and every polarization comes out identical, which looks like a symmetry result. And a field step that moves the Berry phase by more than a quarter of its quantum, since a difference taken across a branch is the quantum divided by δ — large, smooth and meaningless. Everything the polarization refuses is refused here too.

The validation is the spin-orbit null. Without the coupling a global spin rotation maps B to $-B$ and leaves every charge observable fixed, so α vanishes identically. On the same cell, same field, same ultrasoft family at the same valence, with only `lspinorb` and the scalar/fully relativistic datasets changed: 3.43×10^{-9} against the coupled 4.44×10^{-6}, a ratio of **1293**. Two things had to be right together to get there, and neither helps alone: `chain` (seeding one field run from the other, which breaks the very $B \to -B$ symmetry the null rests on — off by default here, unlike Elk) and the *polarization's* threshold rather than the self-consistent field's. **The absolute value is not calibrated against another code**: Elk's `bfieldc` and this package's `B_field` differ in normalisation by about two orders of magnitude, and Elk's own tensor on this cell is not converged.

10 Effective masses, site angular momenta, nesting, structure factors and STM images

Quantities Elk computes and `pw.x` mostly does not. The first two cost one NSCF at a single k-point; the nesting function costs one NSCF on a dense grid, and nothing after it; the structure factors cost one transform of a density that already exists, and an STM image one band sum with different weights.

10.1 The effective mass tensor

$$\left(\frac{1}{m^*}\right)_{ab} = \frac{1}{2}\,\frac{\partial^2 \varepsilon_n(\mathbf{k})}{\partial k_a \partial k_b},$$

in units of $1/m_e$. The factor of a half is Rydberg atomic units and not a normalisation: $\hbar^2/2m_e$ is exactly 1 Ry bohr^2, so a free electron has $\varepsilon = |\mathbf{k}|^2$ and the tensor is the identity.

The first derivative is analytic and only the second is a difference. $\partial\varepsilon_n/\partial k$ is the generalised Hellmann–Feynman expression built from one `jvp` of $H(\mathbf{k})$; its second derivative is not reachable the same way, because an individual band's $|\partial_k \psi_n\rangle$ is not what the

Sternheimer projector produces (it removes the whole occupied manifold, and the band whose mass is wanted is usually empty). So the construction is one central difference of an *exact* first derivative — six stencil points against Elk's twenty-seven, and no polynomial fit. Elk's route, differencing eigenvalues, is kept beside it as `method="eigenvalue"` because it shares nothing with the velocity operator.

```
from defumat import Calculator

calc = Calculator.from_file("si.scf.in")
mass = calc.get_effective_mass((0.0, 0.0, 0.0), nbnd=10)  # crystal coordinates

mass.inverse_mass          # (nbnd, 3, 3) in 1/m_e, cartesian
mass.principal(band=7)     # the three principal masses and their axes
mass.density_of_states_mass(band=7)   # (m_x m_y m_z)^(1/3)
mass.truncation            # the O(h^2) the Richardson step removed
mass.multiplets            # per multiplet, including the degenerate ones
```

On two-atom silicon at Γ against the vendored **Elk** binary (all-electron LAPW, PBE, $a = 10.26$ bohr) with a PAW dataset here: the non-degenerate Γ_{1v} agrees to **0.02%** (0.8600902 against 0.8603044, i.e. $m^* = 1.16267\,m_e$ against 1.16238) and $\Gamma_{2'c}$ to 0.36% ($m^* = 0.170439\,m_e$). The two routes here agree with each other to 1.2×10^{-5}, and the tensors come out isotropic to 2.9×10^{-8} with nothing imposing cubic symmetry.

Refuses. A band inside a **degenerate multiplet** has no tensor of its own — the eigensolver's arbitrary rotation within the manifold rotates it — so those bands come back as `nan` and what is reported instead is the multiplet's *summed* inverse mass, which is the trace and is invariant. Silicon's threefold $\Gamma_{25'}$ is the case, and Elk's own output is the evidence: it prints -5.8938, -3.7690, -3.7690 for three states whose energies agree to 5×10^{-8} Ha. A **spin spiral** is refused, since a k-stencil moves both spheres. `delta` is the stencil's outermost displacement in both routes; the $O(h^2)$ truncation is removed by a Richardson step and then *reported* in `truncation` rather than tuned away.

10.2 $\langle L\rangle$, $\langle S\rangle$ and $\langle J\rangle$ on each atom

Where the orbital moment of a spin-orbit magnet actually sits. From one site density matrix per atom and shell, built out of the same projection a projected density of states uses,

$$\rho^{a}_{(ms),(m's')} = \sum_{n\mathbf{k}} w_{n\mathbf{k}}\, c^{a}_{ms,n\mathbf{k}}\, \overline{c^{a}_{m's',n\mathbf{k}}}, \qquad c = \langle\phi|S|\psi\rangle,$$

and then $\langle L_i\rangle = \sum_s \mathrm{Tr}_m(L_i\rho_{ss})$ and $\langle S_i\rangle = \frac{1}{2}\sum_m \mathrm{Tr}_s(\sigma_i\rho_{mm})$, with $J = L+S$. The L matrices are the complex-basis ones conjugated into the *real* spherical harmonics the code works in, using the same `rot_ylm` unitary the spin-orbit projectors are built from.

```
from defumat import Calculator

calc = Calculator.from_file("ni.scf.in")   # noncolin, lspinorb, nosym
momenta = calc.get_angular_momenta()

print(momenta.table())      # what Elk writes to LSJ.OUT
momenta.atoms[0].l          # <L> on atom 1, cartesian, units of hbar
momenta.atoms[0].j          # <J> = <L> + <S>
```

```
momenta.total_orbital       # summed over the cell
```

Validated by identities, not by another code's number — Elk integrates over a muffin tin of a stated radius and this projects onto an orbital set, so the two differ by definition the way Löwdin and muffin-tin charges do. $\langle L\rangle$ is *quenched to zero* without spin-orbit coupling, measured at 1.7×10^{-16} on silicon; with the coupling on, fully-relativistic nickel gives $\langle L_z\rangle = 0.0364767\,\hbar$ against a measured orbital moment of about $0.05\,\mu_B$, with $|L|/|S| = 0.11665$ against an experimental $m_L/m_S \approx 0.1$ and $\langle L\rangle \parallel \langle S\rangle$ (Hund's third rule). Driving the moment along z, x and y gives $|\langle L\rangle| = 0.0364767$ in all three, with a spread of 7.3e-11 over the three cubic axes, with nothing imposing that a magnitude is a scalar.

Refuses. A **symmetry-reduced k-set**: $\langle L\rangle$ and $\langle S\rangle$ are vectors, and summing a wedge sums a wedge — the axial-vector group average with $\det(R)$ and the time-reversal sign is not written here. Run the whole grid with `nosym = .true.`, unshifted so that it is closed under the point group. A **fully-relativistic ultrasoft or PAW** dataset, because the spinor overlap's off-diagonal spin blocks are `qq_so` and the projection here applies the scalar S to each component; a fully-relativistic *norm-conserving* dataset has $S = 1$ and is exact. And a **spin spiral**, whose two components do not share a set of atomic orbitals. `pw.x`'s `lorbm` computes the *cell's* orbital magnetization and has no per-atom counterpart to check this against — that quantity is the next subsection, and the two are worth reading together.

10.3 The orbital magnetization of the cell

The other half of a magnet's moment, and the half no integral over the unit cell can give. A Bloch state's current circulates through the whole crystal, so $\int \mathbf{r}\times\mathbf{j}$ depends on where the cell is cut — the same difficulty the electric polarization has. What is well defined is a k-space quantity (Thonhauser *et al.*, PRL **95**, 137205 (2005); Ceresoli *et al.*, PRB **74**, 024408 (2006)):

$$\mathbf{M} = \frac{e}{2\hbar c}\,\mathrm{Im}\sum_n \int [d\mathbf{k}]\,\langle\partial_{\mathbf{k}}u_n|\times(H_{\mathbf{k}} + E_{n\mathbf{k}} - 2\mu)\,|\partial_{\mathbf{k}}u_n\rangle,$$

a *local* circulation inside each cell plus an *itinerant* one along the boundary. Neither is separately measurable and their sum is. The derivative is the covariant finite difference over a uniform mesh — the dual states $|w_m\rangle = \sum_n (M^{-1})_{nm}|u_{\mathbf{k}+\mathbf{b},n}\rangle$ of the neighbouring manifold — so nothing is divided by $E_n - E_m$ and the answer is invariant under any unitary mixing of the occupied bands.

```
from defumat import Calculator

calc = Calculator.from_file("i-atom-soc.in")    # noncolin, lspinorb, gapped
orbm = calc.get_orbital_magnetization(divisions=(3, 3, 3))

orbm.lc, orbm.ic     # pw.x's two Kubo terms, Bohr magnetons per cell
orbm.total           # their sum, at mu = 0
orbm.chern           # the Chern vector; zero for an ordinary insulator
```

Against `pw.x`'s `lorbm` on the same $3\times3\times3$ grid, an iodine atom in a twelve-bohr box: $M_{LC} = -0.5986388$ against -0.5986370 and $M_{IC} = -0.5757995$ against -0.5757987, so the total is -1.1744384 against -1.1744357. What is left at 2×10^{-6} is the difference between two

separately converged densities: re-running at `conv_thr` of 1e-6, 1e-8 and 1e-12 moves M_{LC} by 5e-6, 2e-7 and 2e-6 against the same reference. **The physics check is independent of both**: iodine is $5s^2 5p^5$, one hole in the p shell, and Hund's rules give $L = 1$ locked parallel to $S = 1/2$. The site projection of the previous subsection measures $\langle L_z \rangle = 0.99977\,\hbar$ on the same run, and the modern theory gives $1.1744\,\mu_B$ — 17% more, which is what a projection onto atomic orbitals does not see: the circulation outside those orbitals, and the nonlocal pseudopotential's own contribution to the velocity, since $\langle L \rangle$ is $\mathbf{r} \times \mathbf{p}$ where this contracts H.

Two conventions are QE's and are worth knowing. What both codes print is $M(\mu = 0)$: `orbm_kubo` imports the Fermi level and never uses it, and the term it drops is μ times the Chern vector, which vanishes for any topologically trivial manifold. And the two printed terms are *not* the papers' LC/IC split — the Fortran prints $\mathrm{Im}\langle \partial u | H | \partial u \rangle$ and $\mathrm{Im}\langle \partial u | E | \partial u \rangle$, which differ from $(H - E)$ and $2(E - \mu)$ term by term and agree in the sum. This follows the Fortran, so the two halves can be compared and not only the total.

Refuses. Ultrasoft and PAW, which `pw.x` refuses in the same place (`setup.f90`: 'Orbital Magnetization not implemented with USPP/PAW'): the overlap between neighbouring manifolds would need $q_{ij}(b)$ and the dual states would have to be dual in the S metric. A run **without spin-orbit coupling**, where the orbital moment is quenched identically — measured here as 10^{-9} with `soc_scale = 0` in the same relativistic dataset. **`nspin = 2`**, which is the same statement: each channel is separately time-reversal symmetric. A **metal**, or any manifold not separated from the band above it, which the gap check refuses by name. A **spin spiral**. And a mesh with **two divisions** along a direction that carries a derivative, where a point's two neighbours are the same k-point and the central difference is an alias; one division is allowed and sets that direction's derivative to zero, which is what a slab means.

10.4 The Fermi-surface nesting function

How much of the Fermi surface maps onto itself when translated by $\mathbf{q}$:

$$N(\mathbf{q}) = \frac{1}{N_k} \sum_{\mathbf{k}} g(\mathbf{k})\, g(\mathbf{k}+\mathbf{q}), \qquad g(\mathbf{k}) = g_s \sum_{s,n} \delta(\varepsilon_{sn\mathbf{k}} - E_F).$$

$g(\mathbf{k})$ is the density of states *at one k-point* — a picture of the Fermi surface on the grid — so $N(\mathbf{q})$ is the $\omega \to 0$ imaginary part of the Lindhard function stripped of its matrix elements: the *geometric* half of an instability. Where it is large, a perturbation of wavevector $\mathbf{q}$ connects many occupied states to many empty ones at no energy cost, which is what softens a phonon, opens a charge-density-wave gap, or picks a spin spiral's pitch.

It is a convolution. Elk writes an $O(N_q N_k)$ double loop with $\mathbf{k}+\mathbf{q}$ folded back onto the grid; that fold makes the sum a cyclic cross-correlation, so $N = \mathrm{ifftn}(|\mathrm{fftn}(g)|^2)/N_k$ gives every $\mathbf{q}$ in one transform. Measured on a 12^3 aluminium grid that is **0.001 s** here against Elk's 0.37 s. A symmetry-reduced wedge is *unfolded* onto the complete grid rather than refused, since $\varepsilon_n(R\mathbf{k}) = \varepsilon_n(\mathbf{k})$: 72 diagonalisations instead of 1728.

```
from defumat import Calculator

calc = Calculator.from_file("al.scf.in")
nest = calc.get_nesting(grid=(12, 12, 12))   # the grid is the convergence knob

nest.nesting        # (nq,) N(q), in (states/Ry/cell)^2
```

```
nest.qpoints        # (nq, 3) crystal coordinates, on the unshifted grid
nest.ratio          # N(q) / D(E_F)^2, dimensionless and 1 on average
nest.peak()         # the largest N(q) away from q = 0, and where it is
nest.along(2)       # (q_3, N) along one crystal axis, for a line plot
nest.as_grid()      # (n1, n2, n3), for a slice
nest.fermi_dos      # D(E_F) in states/Ry/cell
nest.elk_units      # the same array in NEST3D.OUT's normalisation
```

The functional entry point is `defumat.workflows.run_nesting`, which takes the converged density directly.

Two identities come with the normalisation and both are worth reading. The mean of N over the whole q-grid is exactly $D(E_F)^2$ (`sum_rule` reports the residue; it is 3×10^{-16} relative). And $N(0) \geq N(\mathbf{q})$ for every $\mathbf{q}$ by Cauchy–Schwarz, so **the nesting peak is always a peak away from the origin** — `peak()` excludes $\mathbf{q} = 0$ for that reason, not as a plotting convenience.

Validated against a closed form and against a quantity computed by an unrelated route. For free electrons $N(q) = \Omega/4\pi^2 q$ below $2k_F$ and zero above: the code reproduces the $1/q$ law to 10^{-4} and the hard cut to 10^{-16}. A half-filled hydrogen chain has $2k_F = \pi/c$, so it must nest at $\mathbf{q} = (0, 0, \frac{1}{2})$ — it does, at **99.8%** of the Cauchy–Schwarz bound, and `relax_spiral_q` relaxes the corresponding spin spiral to 0.500014 by going downhill in a magnetic total energy, which shares nothing with this. Against the vendored Elk binary on aluminium, $N(0)/D(E_F)^2$ agrees to 1.5%; the raw $N(0)$ differs by 4.7%, which is the all-electron Fermi surface underneath and is quoted rather than tuned away.

Refuses. A constrained `tot_magnetization`, which has one Fermi level per channel, so $g(\mathbf{k})$ is two different surfaces; a **fixed-occupation** run, which never searches for a level, so $\delta(\varepsilon - E_F)$ has no argument; and a **spin spiral**, because the quantity is a statement about the state a spiral grows out of — compute it on the paramagnet and compare its peak with `relax_spiral_q`'s answer. The delta defaults to a **Gaussian** even when the run used Methfessel–Paxton or cold smearing: those go negative on the wings, so $g(\mathbf{k})$ can be negative and $N(\mathbf{q})$ would have no sign at all. Pass `smearing=` to have the run's own anyway.

10.5 Magnetocrystalline anisotropy

The energy it costs to point a magnet's moment one way rather than another — what makes a hard magnet hard and what pins a spiral into a plane. It is tenths of a meV per atom against total energies of hundreds of Rydberg, so taking it as a difference of two self-consistent total energies asks two SCF runs to agree in their ninth digit. The *force theorem* does not ask that. Converge the magnet **without** spin-orbit coupling, rotate the converged density so its magnetization points along $\mathbf{n}$, and diagonalise **once** with the coupling switched on. At frozen density every term of the total energy except the band energy is a functional of ρ alone, so

$$E(\mathbf{n}_1) - E(\mathbf{n}_2) = E_{\text{band}}(\mathbf{n}_1) - E_{\text{band}}(\mathbf{n}_2), \qquad E_{\text{band}} = \sum_{n\mathbf{k}} w_{n\mathbf{k}}\, \varepsilon_{n\mathbf{k}},$$

exactly, and the whole calculation is one diagonalisation per direction.

The cheaper thing gives zero. Freezing the wavefunctions as well and taking $\langle\psi|H_{\text{SOC}}|\psi\rangle$ once returns no anisotropy at all: the coupling enters at first order as $\xi\,\langle\mathbf{L}\rangle \cdot \mathbf{n}$, and the orbital moment of a scalar-relativistic collinear state is quenched. The anisotropy is second order in the coupling, and what supplies it is the repulsion between levels that a

diagonalisation performs and an expectation value does not. `frozen_expectation` computes that vanishing term anyway.

Two pseudopotential files, one density. The first leg runs with a scalar-relativistic dataset and the one-shot leg with the fully-relativistic dataset of the *same generation*; only the density crosses, and a density does not know which file made it. That is what makes an ultrasoft anisotropy reachable, the alternative — averaging one relativistic file's j channels back — being refused for ultrasoft and PAW by `pw.x` itself.

```
from defumat import Calculator

scalar = Calculator.from_file("scf.in")          # nspin = 2, Co.pbe-nd-rrkjus
spinor = Calculator.from_file("nscf.in")         # noncolin, lspinorb, rel- file
mae = scalar.get_anisotropy(spinor, directions="xyz")

print(mae.anisotropy_mev, mae.easy_axis)
print(mae.difference(0, 2))                      # in-plane minus out-of-plane
```

`directions` takes a list of vectors, an integer asking for that many points spread evenly over the sphere, or one of `"cardinal"`, `"xz"` and `"xyz"`. `projected=True` adds the decomposition of the band energy over atomic orbitals, resolved by atom, ℓ, m and spin, which is what says *which* orbitals supply the anisotropy; comparing its total against E_{band} measures the spilling.

`soc_scale` scales the spin-orbit part of the nonlocal potential and the overlap while keeping the same dataset and the same k-points. At 0 the anisotropy is exactly zero, which is the control for the whole calculation: without the coupling the Hamiltonian is invariant under a global spin rotation, so a band energy *cannot* depend on where the moment points.

The torque: the same number from one angle. Taking the anisotropy as $E(\mathbf{n}_1) - E(\mathbf{n}_2)$ is 10^{-5} Ry out of 10^{2} — seven digits of cancellation. The torque takes it as a *derivative* evaluated once, where nothing cancels: for $E(\theta) = K_1 \sin^2\theta$,

$$-\frac{\mathrm{d}F}{\mathrm{d}\theta} = -K_1 \sin 2\theta, \qquad \text{so at } \theta = 45^\circ \text{ the torque is } -K_1.$$

```
torque = scalar.get_torque(spinor)               # 45 degrees by default
print(torque.anisotropy_constant_mev)            # K1 in meV
```

It differentiates the free energy, and for a smeared metal that is not the band energy. A Hellmann–Feynman derivative at frozen occupations gives $\mathrm{d}F/\mathrm{d}\theta$ with $F = \sum w\varepsilon - TS$; $\sum w\varepsilon$ carries an extra $\sum(\mathrm{d}w/\mathrm{d}\theta)\varepsilon$ that the entropy cancels. The difference is not small. On tetragonal cobalt, as the smearing narrows:

`degauss` (Ry)	band difference	free difference	torque
0.020	1.2353	0.5523	0.5523
0.005	0.5434	0.5291	0.5291
0.002	0.5308	0.5308	0.5308

all in meV. The torque tracks the free-energy difference at every width and both converge on $K_1 = 0.531$; at a production `degauss` of 0.02 the *band*-energy difference reads 2.3 times that. Use `MagneticAnisotropy.free_energies` to compare like with like.

Refuses. **PAW**, because the handoff from the collinear run carries the density and nothing else, and a PAW Hamiltonian needs `ddd_paw`, which is built from a `becsum` belonging to a run with a different pseudopotential file (`pw.x` refuses it in the same place). A **Hubbard** U, whose `ns` is not in the handoff either; a potential-only **meta-GGA**, whose τ is not; a converged **magnetic field** or constrained moment, whose energy sits outside the reported total; and a **spin spiral**, which refuses spin-orbit coupling permanently and so has no coupling to switch on. A direction other than the system's own `angle1`/`angle2` needs `nosym`: a magnetic noncollinear run reduces its k-grid with the *magnetic* symmetry group, which depends on where the moment points, so two directions would otherwise be sampled on two different wedges. `soc_scale` takes 0 or 1 and nothing in between: the coupling-free end of the overlap is spin-independent, so the anisotropy vanishes identically there whatever else it is, but a blend of it with the true overlap partway is not the overlap of any set of projectors and S stops being a usable metric.

10.6 X-ray and magnetic structure factors

What a diffractometer measures is not a density but its Fourier coefficients, one per reflection:

$$F(\mathbf{H}) = \int_\Omega \rho(\mathbf{r})\, e^{i\mathbf{H}\cdot\mathbf{r}}\, d^3r, \qquad F_j(\mathbf{H}) = \int_\Omega m_j(\mathbf{r})\, e^{i\mathbf{H}\cdot\mathbf{r}}\, d^3r$$

for every reciprocal lattice vector with $|\mathbf{H}| \le$ `hmax` — X-rays scattering off the charge, neutrons off the magnetization. Elk's tasks 195 and 196; `pw.x` has neither. In a plane-wave code the integral *is* an array that already exists, up to the crystallographers' two conventions (the positive phase, and no $1/\Omega$), so the whole step is one transform and a gather: **6 ms** against the seconds its SCF cost. $F(0)$ is the electron count of the cell and $F_j(0)$ its magnetic moment, which is what anchors both.

```python
from defumat import Calculator

calc = Calculator.from_file("si.scf.in")
factors = calc.get_structure_factors(hmax=5.0)

factors.charge               # (nh,) complex F(H), in electrons
factors.magnetization        # (nh, ncomp) in Bohr magnetons, or None
factors.vectors.indices      # (nh, 3) the (h k l) after the output transform
factors.vectors.length       # (nh,) |H| in 1/bohr
factors.vectors.multiplicity # the size of each star that was collapsed
factors.of((2, -2, -2))      # one reflection, by the index the table prints
print(factors.table(limit=10))

# Elk's wsfac: rebuild the density from the states in an energy window (Ry),
# which is what makes this a probe of bonding rather than of the whole charge.
bonding = calc.get_structure_factors(hmax=3.0, window=(-1.0, 0.2))
```

The functional entry point is `defumat.workflows.sfac.run_structure_factors`. `transform="conventional"` labels the reflections on the conventional cubic axes (Elk's `vhmat`), `reduce=False` keeps every member of every star, and `core=True` adds a dataset's frozen core-correction charge.

A pseudopotential density is valence-only, and the comparison with an all-electron code is structured by reflection class rather than by one tolerance. On silicon, against the vendored Elk binary on the same cell and k-grid: the origin is 8 electrons here and 28 there, and every *allowed* reflection is core-dominated in the same proportion

— (111) is 1.7495 against 15.14, and no tolerance makes that agree. What *is* comparable is a **forbidden** reflection, where the spherical part of every atom cancels and the core with it: silicon's (222) is **0.347406** here against **0.33416**, 4%, on a quantity whose entire value is the aspherical bonding charge. That last statement is checked without any other code, since this package's own SCF starting guess is a superposition of free atoms: it gives 0 at (222) to 10^{-10}. The space-group forbidden reflections — diamond's (200), (420) — vanish to 10^{-9} here and 10^{-13} there.

Norm-conserving, ultrasoft and PAW all work, and the difference between them is measurable. $F(0) = 8.0000000000$ on each, which is the check that the augmentation charge reached the transform at all — for a soft dataset most of the density near the nucleus *is* that charge. The like-for-like reference is not Elk's all-electron total but its **valence-only** run, which `wsfac` above the 2p core produces: against it the augmentation charge is worth a factor of two, (111) coming out -0.7% for ultrasoft and PAW against -1.5% norm-conserving, and (004) $+31$% against $+61$%. It is worth *nothing* on the forbidden (222), which is 0.3474 on all three to 3×10^{-4}: a silicon atom in diamond sits at $\bar{4}3m$, whose lowest non-spherical invariant is $l = 3$, and an s, p dataset's augmentation charge carries multipoles only to $L = 2$. That is also what the 4% against all-electron is, by elimination — not the k-grid, not either basis, not the functional (0.07%), and **not the core**, since Elk's own valence-only run gives the identical 0.334165.

Magnetic reflections are a low-$|\mathbf{H}|$ quantity and the boundary is measured. Ferromagnetic bcc iron against Elk's task 196, comparing the normalized $F_{\mathrm{mag}}(\mathbf{H})/F_{\mathrm{mag}}(0)$ because the moments differ (2.2145 μ_B here, 2.0613 there): 5.8% at the first reflection, 17% at the second, worse outwards. Raising `ecutwfc` from 30 to 45 moves those by less than 0.1%, so it is the pseudisation and not the basis — iron's moment is in the 3d shell, inside the radius the dataset smooths. The cheapest cell carries the sharpest magnetic statement instead: an antiferromagnetic hydrogen chain has charge factors $2.000000, 0, 0.8587, 0$ along the chain and magnetic ones $0, 1.2254, 0, 0.3862$, so it scatters neutrons exactly where it scatters no X-rays, and the zeros are exact.

Refuses. An `hmax` past $\sqrt{\texttt{ecutrho}}$: the FFT box carries a coefficient at every frequency out to its corner, and past the density's own cutoff those are the aliased tail of $|\psi|^2$ rather than Fourier components — small, smooth and entirely plausible, which is why this is a guard and not a check on the answer. An empty or inverted energy window, and `core=True` on a dataset with no nonlinear core correction. **Said rather than refused**, because the numbers are right and only a claim about them would be wrong: these are *valence* structure factors, and a star's multiplicity applies to the charge, the magnetic members of a star being related by $\det(R)\,R$ rather than being equal.

10.7 Scanning-tunnelling microscopy images

Tersoff and Hamann's result is that the current an s-wave tip draws at $\mathbf{r}$ is the sample's local density of states there, at the energy the bias selects. So an STM image is not a new calculation, it is the *density built from different occupations*:

$$\rho_{\mathrm{STM}}(\mathbf{r}) = \sum_{n\mathbf{k}} w_{\mathbf{k}}\, \tilde{\delta}(E - \varepsilon_{n\mathbf{k}})\, |\psi_{n\mathbf{k}}(\mathbf{r})|^2,$$

with $\tilde{\delta}$ the same smeared delta a density of states is built from. Elk's task 162 and `pw.x`'s `plot_num = 5` both say exactly that. Two energy selections: with no `bias` the weight is a delta at E (Elk's, a density of states per unit volume, in $1/(\mathrm{bohr}^3\,\mathrm{Ry})$), and with one it is

the window $[E, E+V]$ (QE's, a density in electrons/bohr^3) — a *negative* V imaging the filled states, which is a real experiment's sign convention and the mode a semiconductor surface needs.

```
from defumat import Calculator

calc = Calculator.from_file("graphene.scf.in")

# constant height: the tip plane at crystal coordinate 0.85 along c
image = calc.get_stm(height=0.85, shape=(64, 64))

image.values        # (64, 64) the tunnelling density on the plane
image.coordinates   # (64, 64, 2) in-plane cartesian coordinates, bohr
image.extent()      # (x0, x1, y0, y1) for imshow
image.integral      # int rho_STM d3r = D(E_F): the sum rule
image.density       # the same field on the FFT grid

# filled states, 0.15 Ry below the Fermi level
filled = calc.get_stm(height=0.85, bias=-0.15)

# constant current: the corrugation, in bohr along the surface normal
scan = calc.get_stm(height=0.70, mode="constant-current",
                    current=1.0e-5, heights=(0.0, 6.0), nheights=80)
scan.heights_bohr   # (n1, n2), nan where the set-point is not crossed

# a magnetic tip: the image depends on which way it points
up = calc.get_stm(height=0.85, spin="up")            # collinear
along_x = calc.get_stm(height=0.85, spin=(1, 0, 0))  # noncollinear

# Elk's three corners, in crystal coordinates, for any other plane
side = calc.get_stm(plane=((0.15, 0, 0), (0.15, 0.3, 0), (0.15, 0, 1)))
```

The functional entry point is `defumat.workflows.stm.run_stm`. A delta at the Fermi level wants a denser k-grid than the SCF's, which `grid=` re-solves the bands on and **recomputes the Fermi level for**; `smearing` picks the delta (a Gaussian by default whatever the run used, because Methfessel–Paxton and cold smearing go negative on their wings and a negative tunnelling current is meaningless); and `band_cutoff` is `stm.f90`'s fixed truncation at three widths, off by default because it is an approximation.

The plane is read out of the grid exactly. A density is a finite sum of plane waves, so its value at a point between grid points is that sum evaluated there rather than a spline through neighbours. It matters for the one thing an image is about: the corrugation in the vacuum is a small modulation on a quantity falling by orders of magnitude across one grid spacing, and an interpolant of it carries the grid's own periodicity as a false corrugation. The three corners are in *crystal* coordinates, Elk's `vclp2d`, and the sampling excludes the far edge (i/n, not $i/(n-1)$) so a whole-cell image tiles.

Against `pp.x`: 6.7×10^{-10} on every point of the 15^3 grid of QE's own fcc aluminium, reading its `filplot` text dump so that nothing is interpolated on either side. Three of QE's conventions are needed to make it the same sum: `stm.f90` divides by the volume and not by `degauss`, so its image is the zero-bias one times the width; it uses the run's own smearing; and it truncates the band sum at three widths. That truncation is worth 0.4% *and the complete sum is the smaller one* — Marzari–Vanderbilt's delta is negative for $x > \sqrt{2}$, so the states QE drops carry negative weight. The check that involves no other code is the sum rule $\int \rho_{\mathrm{STM}}\, d^3r = D(E)$ against `compute_dos`, which holds to 10^{-10} on a scalar run and on a spinor one, and in its window form on an ultrasoft one — the augmentation charge follows the

tunnelling weights, which is why an ultrasoft or PAW image works here and `stm.f90`'s does not.

A magnetic tip is the part neither code has. It counts the states whose spin is along its own moment, so with P the tip's polarization

$$\rho_{\mathrm{STM}}(\mathbf{r};\hat{\mathbf{n}}) = \tfrac{1}{2}\left[\rho(\mathbf{r}) + P\,\hat{\mathbf{n}}\cdot\mathbf{m}(\mathbf{r})\right],$$

and $P = 1$ makes the image a genuine spin channel. On an antiferromagnetic hydrogen chain the charge image is flat to six figures while the two spin projections are each other's mirror ($\pm 2.58\%$, antisymmetric to 10^{-3}), and they add back to the charge to 10^{-14}. On a chain whose four moments each turn 90 degrees from the last, a tip along x sees atoms 1 and 3 with opposite sign and atoms 2 and 4 **not at all** (10^{-16} against a contrast of 0.063), while a tip along z sees none of the four — with nothing imposing any of it.

The physics case is graphite. In AB-stacked bilayer graphene one sublattice of the surface layer sits above an atom of the layer below and the other above a hollow, so the two are inequivalent and only one is bright at the Fermi level: $1.64\times$ here, on two atoms of the same element carrying the same charge. That is the textbook result that half the atoms of graphite are invisible to an STM, and it is a statement about the density of states *at the Fermi level* rather than about the charge. The tunnelling density falls off log-linearly into the vacuum ($R^2 = 0.9992$ over five orders) at a rate of at least $2\sqrt{V_{\mathrm{vac}} - E_F}$, which is Tersoff–Hamann's own statement and involves none of the assembly; the bound is one-sided because graphene's states at E_F sit at K, whose in-plane momentum steepens the decay (measured 1.86, against 0.98 for $k_\parallel = 0$).

Refuses. A spin spiral (its two spinor components live on different plane-wave spheres, so $|\psi(\mathbf{r})|^2$ is not the lattice-periodic object this sum builds), a constrained `tot_magnetization` (one Fermi level per channel, and a tip sees one energy), an applied magnetic field (its energy is outside the reported total, so the level the tip would be put at is not the field-free one the image would be read as), a spin direction on a run with no magnetization, and a *transverse* direction on a collinear run — there m_x is absent rather than zero, and projecting onto the axis would be a claim the calculation cannot make. A constant-current scan longer than the lattice period along the plane's normal, where the tip meets the periodic image of the surface and the density rises again. **Said rather than refused**: a constant-current image returns `nan` at every point where the set-point is not crossed inside the scan, with a warning saying how many; and a zero-bias image of a gapped run is identically zero, the delta sitting in the gap, which is what `bias` is for.

10.8 Vertical tunnelling transport through a two-dimensional material

An STM image says what the tip *sees*. This says what gets *through*: an electron enters at a point $\mathbf{r}$ above the material and leaves into an infinite plane below it — the substrate the sheet sits on — so what decides the current is the *nonlocal* Green's function between the two rather than the local density of states at the tip. A **point** tip makes Γ_t rank one, which collapses the Landauer–Büttiker trace exactly:

$$T(\mathbf{r};E) \;=\; \mathrm{Tr}[\Gamma_t G^r \Gamma_s G^a] \;\propto\; \int_{\mathrm{plane}} \mathrm{d}^2 r' \,\left|G(\mathbf{r},\mathbf{r}';E)\right|^2 .$$

The substrate is taken to be *infinite and featureless*, so it is invariant under every lateral lattice translation and conserves the lateral momentum. The exit integral is then diagonal

in $\mathbf{k}$, and everything reduces to one quadratic form per k-point,

$$T(\mathbf{r};E) = \sum_{\mathbf{k}} w_{\mathbf{k}} \sum_{nm} a_{n\mathbf{k}}(\mathbf{r})\, a^*_{m\mathbf{k}}(\mathbf{r})\, S_{\mathbf{k}}[n,m], \qquad S_{\mathbf{k}}[n,m] = \int_{\text{plane}} \psi^*_{n\mathbf{k}} \psi_{m\mathbf{k}}\, \mathrm{d}^2 r',$$

with $a_{n\mathbf{k}} = \psi_{n\mathbf{k}}(\mathbf{r})\sqrt{\tilde{\delta}(E-\varepsilon_{n\mathbf{k}})/\eta}$. So **the sum over k is incoherent and the interference is between bands at the same k**, which is what "the substrate is featureless" means physically. $S_{\mathbf{k}}$ is a Gram matrix, so the transmission is non-negative by construction and is blind to a rotation inside a degenerate multiplet.

Either electrode can be magnetic. A spin-polarized substrate is a 2×2 acceptance $P_s = (1 + P_s\, \hat{\mathbf{n}}_s \cdot \boldsymbol{\sigma})/2$ sitting *inside* $S_{\mathbf{k}}$, between two bands. A spin-polarized *tip* is the same matrix on the other side, $\Gamma_t = \gamma_t P_t$, and it is not a weight but a contraction: with the spinor Green's function $G_{ss'}(\mathbf{r},\mathbf{r}') = \sum_n a_{n,s}(\mathbf{r})\, \psi^*_{n,s'}(\mathbf{r}')$ the Landauer trace is no longer taken over the tip's spin, and the two spinor components stop adding and start interfering,

$$T(\mathbf{r}) = \int_{\text{plane}} \mathrm{d}^2 r'\, \mathrm{Tr}_{\text{spin}}\left[\Gamma_t\, G\, \Gamma_s\, G^\dagger\right] = \sum_{\mathbf{k}} w_{\mathbf{k}}\, \mathrm{Tr}_{\text{spin}}[P_t\, M_{\mathbf{k}}(\mathbf{r})], \qquad M_{\mathbf{k}}[s,s'] = \mathbf{a}_s^{\mathsf{T}}\, S_{\mathbf{k}}\, \mathbf{a}^*_{s'}.$$

M is itself a Gram matrix, $A^{\mathsf{T}} S A^*$ for the $(n_{\text{bnd}}, 2)$ amplitude block, so it is positive semi-definite and $\mathrm{Tr}[P_t M] \geq 0$: the non-negativity survives. With *both* leads polarized the map depends on the angle between the two moments, which is a **tunnelling magnetoresistance** image. **Note the asymmetry of the names**: here `spin` is the *substrate* and `tip_spin` the *tip*, which is the opposite way round from `get_stm`, where `spin` is the tip. The substrate was the polarizer first and the name was kept.

```
from defumat import Calculator

calc = Calculator.from_file("graphene.scf.in")

# tip plane above the sheet, substrate plane below it, atoms in between
t = calc.get_vertical_transport(exit_height=0.38, height=0.62,
                                shape=(48, 48), broadening=0.02,
                                grid=(6, 6, 1))

t.image            # (48, 48) the transmission map, arbitrary units
t.interference     # coherent minus incoherent: what no local picture gives
t.notes["channels"]  # open transmission channels the substrate offers
t.extent()         # (x0, x1, y0, y1) for imshow

# a transport dI/dV at one place: the tip sampling is paid once
import numpy as np
spectrum = calc.get_vertical_transport(
    exit_height=0.38, tip=np.array([[0.0, 0.0, 0.62]]),
    energies=calc.get_scf().fermi_energy + np.linspace(-0.2, 0.2, 41),
    broadening=0.02, grid=(6, 6, 1))

# a spin-polarized substrate: only on a magnetic run, and the two
# directions partition the unpolarized answer exactly
magnet = Calculator.from_file("h-sheet-lsda.scf.in")
up = magnet.get_vertical_transport(exit_height=0.15, height=0.85,
                                   spin="up", broadening=0.05)
down = magnet.get_vertical_transport(exit_height=0.15, height=0.85,
                                     spin="down", broadening=0.05)

# a magnetic tip on a noncollinear run, and both electrodes at once:
```

```
# the map then measures the angle between the two moments
spinor = Calculator.from_file("h-sheet-noncolin.scf.in")
valve = dict(exit_height=0.15, height=0.85, broadening=0.05,
             shape=(24, 24), spin=(1.0, 0.0, 0.0), polarization=0.9)
parallel = spinor.get_vertical_transport(tip_spin=(1.0, 0.0, 0.0),
                                         tip_polarization=0.9, **valve)
antiparallel = spinor.get_vertical_transport(tip_spin=(-1.0, 0.0, 0.0),
                                             tip_polarization=0.9, **valve)
tmr = antiparallel.image.mean() / parallel.image.mean()   # 5.29 on this cell

parallel.tip_spin           # (1.0, 0.0, 0.0), beside .spin for the substrate
parallel.tip_polarization   # 0.9
```

The amplitude is the on-shell one and that is a measurement, not a preference. The literal Landauer denominator $1/(E-\varepsilon+i\eta)$ cannot be evaluated by a sum over states: the states far from E build the barrier's evanescent decay entirely by cancellation between themselves. On a cell small enough to diagonalise completely (367 plane waves, 367 exact bands) the running sum for one $G(\mathbf{r},\mathbf{r}')$ wanders over more than an order of magnitude and lands only at the complete basis, with a cancellation ratio of **349**. Replacing the denominator by its modulus — which is Bardeen's golden rule, the sample visited on shell — keeps the interference between bands degenerate at the tip energy, which is the interference a real experiment lets happen, and converges: to **3e-7** in the mean between 8 and 45 bands. `method="resolvent"` is reachable and warns.

Validated by identities, since neither `pw.x` nor Elk computes this. (QE's `pwcond.x` is a Landauer transmission of a different geometry: two semi-infinite crystalline leads with the current along one axis, one conductance per energy, and no point contact and so no map. Elk has none.) Widening the substrate from a plane to the whole cell makes $S_{\mathbf{k}}$ the identity by orthonormality, so the quantity becomes the Tersoff–Hamann tunnelling density of states *exactly* — agreeing with `run_stm` to **4.6e-13** with no factor between them, which is the check a normalisation error could not hide in. $S_{\mathbf{k}}$ swept along its own normal integrates to the identity to 1.4e-15, and agrees with a real-space quadrature to 5e-16. A substrate polarized along $\hat{\mathbf{n}}$ plus one along $-\hat{\mathbf{n}}$ is a substrate that takes both, to round-off; one across the moment takes exactly half. The same holds for the *tip*, to **3.1e-16**, with the substrate polarized along a third direction at the same time, and a tip at $P=0$ is exactly half the unpolarized map (1.5e-16), the convention a nonmagnetic tip has in the STM images. A *magnetic* tip in the whole-cell limit reproduces the spin-polarized tunnelling density of states to **5.8e-13**, on a sheet whose moment is put in a generic direction: the transposed index order of M would return the answer for a tip at $(n_x,-n_y,n_z)$, which is real, non-negative and — on a moment lying along an axis — identical, so the check is only a check with $m_y\neq 0$. On that cell the mirrored tip's image differs by a factor of two. Both electrodes polarized at $P=0.9$ give a parallel-to-antiparallel contrast of a factor **5.3**, with a perpendicular tip at the mean of the two to 5e-6. The same cell run as $n_{\mathrm{spin}}=1$, $n_{\mathrm{spin}}=2$ and as a spinor agrees to 1e-10 and 1e-5. And the physics: graphene's two Dirac states at K are degenerate partners, so the symmetry of that point makes the substrate's overlap on the pair a multiple of the identity and leaves nothing off-diagonal for the current to interfere through; its map correlates with the STM image at **1.000000** with zero interference, while an AB bilayer — whose current must cross both layer-polarized sheets — falls to **0.16**, its coherent map a factor of 26 below the incoherent one.

Refuses. A k-set with more than one division along the stacking axis: the lateral momentum is conserved exactly and the perpendicular one is not, so two $k_\perp$ at the same $k_\parallel$ interfere with a phase that depends on where the exit plane sits. A *symmetry-reduced* k-set, because a wedge sums to the map symmetrised over the whole point group while only the subgroup that leaves the exit plane in place belongs to this geometry — a mirror through the slab exchanges the tip side with the substrate side; `grid=` builds the whole grid for that reason. A tilted exit plane, which has no closed-form in-plane orthogonality to stand on. A tip or exit plane *inside* an augmentation sphere on an ultrasoft or PAW dataset — in the vacuum a pseudo-wavefunction is the true one, which is why those datasets otherwise need nothing extra. A spin-selective substrate *or a spin-polarized tip* on a run with no magnetization, there being nothing for either moment to couple to and every direction taking half of everything, which is the charge map again; and a transverse direction on a collinear run for either lead, where m_x and m_y are absent rather than zero. And, inherited from the STM images unchanged: a spin spiral, a constrained `tot_magnetization`, an applied magnetic field. **Said rather than refused**: a geometry whose atoms do not lie between the two planes warns — a cell is periodic, so with both planes on the same side the electron tunnels through the vacuum and around the periodic image, which is a real number and not this one. **Not claimed**: a finite contact patch (which breaks the k-diagonality), a tip with structure beyond an s-wave, and an absolute conductance — the tip and substrate couplings are unfixed prefactors, so what the result carries is the map and its contrast.

11 Things with no `pw.x` counterpart

Spin spirals by the generalized Bloch theorem: a spiral of wavevector $\mathbf{q}$ is a spinor whose two components live on *different* plane-wave spheres,

$$\psi_{\mathbf{k}}(\mathbf{r}) = \begin{pmatrix} e^{i(\mathbf{k}-\mathbf{q}/2)\cdot\mathbf{r}}\, u^{\uparrow}(\mathbf{r}) \\ e^{i(\mathbf{k}+\mathbf{q}/2)\cdot\mathbf{r}}\, u^{\downarrow}(\mathbf{r}) \end{pmatrix}$$

so in the rotated frame the density is lattice periodic and the SCF, the functional and the mixer are untouched. Only two terms carry $\mathbf{q}$ — $|\mathbf{k} \mp \mathbf{q}/2 + \mathbf{G}|^2$ and v_{nl} — and $dE/d\mathbf{q}$ at frozen coefficients is `jax.grad` of them.

```
from defumat.workflows.spiral import (run_spiral_scan, relax_spiral_q,
                                      heisenberg_exchange)

scan = run_spiral_scan(system, pseudos, wavevectors)   # E(q)
best = relax_spiral_q(system, pseudos)   # BFGS on the reciprocal metric
print(best.wavevector, best.scf.total_energy)

J = heisenberg_exchange(scan, system.cell, shells)   # fit E(q) for the J(R)

scan = run_spiral_scan(system, pseudos, wavevectors, gradients=True)
print(scan.relative)     # E(q) - E(q_0) in mRy, from the energies
print(scan.integrated)   # the same curve, from dE/dq along the path
```

`heisenberg_exchange` fits $E(\mathbf{q}) - E(0) = m^2 \sum_{\mathbf{R}} J(\mathbf{R})\,[1 - \cos(\mathbf{q}\cdot\mathbf{R})]$ over a set of lattice shells, which is how a spiral scan becomes a spin model. `compute_spiral_gradient` gives $dE/d\mathbf{q}$ on its own.

$E(\mathbf{q})$ two ways. With `gradients=True` the scan also takes $dE/d\mathbf{q}$ at every point, and `scan.integrated` accumulates it along the path,

$$E(\mathbf{q}_n) - E(\mathbf{q}_0) = \sum_m \int \mathrm{d}\mathbf{q} \cdot \frac{\partial E}{\partial \mathbf{q}},$$

by the trapezoid rule on the segments between successive wavevectors (both factors in lattice coordinates, so the contraction needs no metric and the path may bend). The two curves are the same physics and the gap between them is the useful part: the energies are computed on a plane-wave sphere *rebuilt* at every point and step by the Pulay error of a finite basis wherever a plane wave crosses the cutoff, while the gradient is taken at a *frozen* sphere and does not see those steps. On the hydrogen chain of `tests/data/qe/h-chain-spiral.in` at `ecutwfc = 25` the direct energies rise, fall and rise again over a 13-point scan where the integrated curve descends monotonically; the two agree to 0.005 mRy at `ecutwfc = 60` once the quadrature error is removed, against 0.130 at 25.

Two error sources live in that gap and refining tells them apart: the trapezoid rule's own h^2 shrinks when the **q** path is refined ($0.051 \to 0.016 \to 0.003$ mRy for $7 \to 13 \to 25$ points) and the basis noise does not (0.139, 0.138, 0.137). What the integrated route does *not* buy is worth stating, since the intuition that it might is a common one: it is not cheaper (every point still needs its own SCF), it does not converge on a coarser k-mesh (the gradient is the exact derivative of the *same* fixed-mesh energy, so it inherits the same Brillouin-zone error), and it needs a *tighter* `conv_thr` rather than a looser one, an energy's error being second order in the density's where a derivative's is first.

Also here: **per-atom magnetic fields**, the **topological invariants** of the previous section, and **rank-n tensor symmetrisation** — `symme.f90`'s `symmatrix3`/`symtensor3` written at any rank.

Refuses. A spiral with spin-orbit coupling, permanently — it breaks the theorem, and Elk refuses it too; a spiral with symmetry, until the spin space group is written, so a spiral needs `nosym`; a spiral with ultrasoft/PAW; and spiral relaxation under a magnetic field, whose energy is outside the reported total, so the state is stationary for a different functional than the one being differentiated.

12 Continuing an all-electron ground state

Elk is an all-electron LAPW code and this is a pseudopotential plane-wave one, so their converged densities are different objects: Elk's carries the core electrons and the nuclear cusp, and this one carries a smooth pseudo valence density normalised to the valence electron count. What crosses between them is a *starting guess*, and the run's own SCF still happens on top of it.

Inside each muffin tin Elk stores the density as a real-spherical-harmonic expansion on a logarithmic radial mesh, and everywhere else as a periodic function on a uniform grid,

$$\rho(\mathbf{r}) = \begin{cases} \sum_{lm} f_{lm}(r)\, R_{lm}(\hat{\mathbf{r}}), & |\mathbf{r} - \mathbf{R}_a| < r_a, \\ \sum_{\mathbf{G}} \tilde{\rho}(\mathbf{G})\, e^{i\mathbf{G}\cdot\mathbf{r}}, & \text{otherwise}, \end{cases}$$

so putting it on a plane-wave grid means deciding which region each grid point is in, then interpolating radially or summing the Fourier series. The muffin-tin part is written in pointwise and is never Fourier truncated, which is why the nuclear cusp transfers exactly and why nothing is lost to aliasing.

```python
from defumat import Calculator

calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")
seed = calc.get_elk_seed("elk_run/")     # STATE.OUT beside GEOMETRY.OUT
result = calc.get_scf(starting_density=seed)
```

The argument is an Elk *run directory* rather than a file: `STATE.OUT` carries neither the lattice vectors nor the atomic positions, so `GEOMETRY.OUT` has to be beside it. And `elk.in` is not a substitute, because Elk's default `tshift` moves the origin onto the inversion centre and only `GEOMETRY.OUT` is written in that shifted frame. Pass `report=[]` to get back a `SeedReport`, which is what the reconstruction measured on the way: what it integrated to before renormalisation, how many of Elk's reciprocal lattice vectors this run's density cutoff kept, and what fraction of the norm sat outside it.

`ElkState` is the reader on its own, for looking at the state rather than seeding with it. `read_elk_state` is the same thing as a function and `read_elk_geometry` reads `GEOMETRY.OUT` alone, returning the `ElkGeometry` that `state.geometry` also holds; all four are exported from `defumat.io`.

```python
from defumat.io import ElkState

state = ElkState.read("elk_run/")
print(state.rmt, state.efermi)            # muffin-tin radii; e_F in Rydberg
print(state.muffin_tin_charges())         # the charge inside each sphere
print(state.interstitial_charge())        # and the charge outside all of them
rho = state.evaluate_at(points)           # the density at arbitrary points
```

The pieces of the reconstruction are exported too, from `defumat.io.elk_density`, because each is a separately checkable part of somebody else's convention rather than an internal: `real_spherical_harmonics` (the sign convention is Elk's and is not the textbook one), `poly4` (the 4-point Lagrange radial window), `spline_weights` (the quadrature the radial integrals use, where Simpson's rule is 9e-6 away), `characteristic_function` (the Fourier-truncated step that separates the two regions) and `interstitial_coefficients` (`rhoir` as a G-vector list). `density_on` is what `get_elk_seed` calls.

What it is worth, measured. On simple-cubic hydrogen at $a = 3.0$ bohr, the reconstruction agrees with Elk's own evaluation of its own density at every one of the 4096 points Elk printed, to 4.5×10^{-11}, which is the precision the file was printed at rather than any error in the transfer. The charge inside the sphere is 0.6125761996 and outside it 0.3874238004, both exact to every digit Elk printed. And a run started from that density and a run started from a superposition of atomic charges converge to the same state: -1.080181442650 Ry either way, agreeing to 6×10^{-14} Ry with converged densities 9×10^{-8} apart.

It buys no iterations, and that is the honest result. Both runs take four. At a lower cutoff the seeded one takes *one more*. For a hydrogen atom a superposition of atomic charges is already almost the answer, and the all-electron cusp inside the pseudisation radius is a place where Elk's density is further from the pseudo one than the atomic guess is. The reader is worth having because it bridges to an all-electron ground state, not because it is faster.

Refuses. A **spin-polarized** Elk state, and a **spin spiral** with it — a magnetization is a second field with a transfer rule of its own, and a seed that carries the charge while silently dropping the moment would start a magnetic run from a non-magnetic

guess. A **DFT+U** or fixed-tensor-moment state, whose occupation matrices are trailing records nothing here reads. A state written by Elk older than **2.0.0**, which Elk's own reader refuses. A directory with no `GEOMETRY.OUT`, or one whose geometry is from a different run. A calculation on a **different cell** — a density is transferable between identical lattices and there is no interpolation here that would make anything else mean something. A calculation with more than one spin channel. And an element with a **core**: the muffin-tin density includes the core states and nothing in `STATE.OUT` separates them from the valence charge, so a run whose valence electron count is far from what the state integrates to is refused by name rather than scaled to fit. Hydrogen, with no core at all, is what this path is for.
Passing an Elk density as a **fixed** density, with no SCF above it, is not offered: the two codes' converged densities are different functions wherever there is a core, and no splice of the two would be a solution of anything.

13 Performance: dials, memory and GPUs

Two batching dials control how much is in flight, and both defaults follow the platform: on a CPU, QE's loop — one k-point and one band at a time, which is what a *cache* wants — and on a GPU the whole of both axes, which is what a card wants. Either end is reachable everywhere:

```
DEFUMAT_K_BATCH=all DEFUMAT_BAND_BATCH=all python3 run.py
```

or per call, `run_scf(..., k_batch=...)`. The chunk size changes the order contributions are summed in and nothing else — round-off, ∼1e-15 Ry.

The band dial is worth twice as much in a spinor run, because a band in flight is then *two* real-space boxes rather than one: a noncollinear calculation with 403 bands on a $240 \times 54 \times 200$ grid puts $403 \times 2 \times 2{,}592{,}000$ complex numbers — 33 GB — into one transform if the whole block goes at once, and several such arrays are live inside a single application of H. `DEFUMAT_BAND_BATCH=16` makes that 1.3 GB. The answer is unchanged bit for bit, not merely to round-off, because chunking the band axis reorders nothing: it is a map, not a sum.

Why the default is per platform. A GPU has no cache to fit in and inverts the conclusion: on a ten-atom metal, `k=1,b=1` costs **4.5x** what `k=all,b=all` does on the same card, sixteen-atom silicon runs at **0.20x** — an outright loss against four CPU cores — and at thirty-two atoms `b=1` did not finish at all. So a run on a device gets both batched unless it says otherwise, and the plausible half-measure `k=all,b=1` is never the default: it is the *worst* setting available, because it buys the batched mode's memory with the looped mode's kernel launches. An explicit argument beats the environment variables, which beat the platform.

Measured GPU speedups (one H200 against one CPU core, per SCF iteration): 2x on a cheap metal, 3–5x on norm-conserving silicon, 9–21x with an augmentation charge or a magnetization, and **83–115x on spin-orbit**. Energies agree to ∼1e-9 Ry — the same agreements the CPU already has.

Memory. Spin-orbit is where it bites: a 20-atom bismuth cell with a relativistic ultrasoft dataset peaks at **34.7 GB**, which fits a 141 GB card and does not fit a 32 GB one. Everything else in that sweep sits under 2 GB.

A response costs memory according to how it is differentiated, which is worth

knowing before starting one. A *forward* response — the Sternheimer solves behind ϵ and Z^* — costs 0.7–1.0 GB over the SCF that produced the state, and a *forward-over-reverse* one — the dynamical matrix, the Raman tensors — 1–2 GB. The *stress* is the exception and it is a *reverse* derivative through the radial transforms: **10.5 GB** on eight-atom ultrasoft silicon, ten times any of the others, and an order more than the SCF it came from. Measure your own with `tools/gpu/phase5.py <case> --property <name>`.

The augmentation charge is the largest object in an ultrasoft or PAW run on a large cell, and there are two ways of keeping it. Below `DEFUMAT_AUG_MAX_BYTES` (default 2 GB) $Q_{ij}(\mathbf{G})$ is built on the whole G sphere once and kept, which is what every number in the accuracy table below was measured with. Above it, Quantum ESPRESSO's scheme takes over: the *radial* transforms

$$Q^L_{nm}(q) \;=\; \frac{4\pi}{\Omega}\int_0^{r_{\mathrm{kkbeta}}} \mathrm{d}r\; j_L(qr)\,\left[r^2 Q^L_{nm}(r)\right]$$

are tabulated on a grid in $|q|$ at $\mathrm{d}q = 0.01$ and $Q_{ij}(\mathbf{G})$ is rebuilt from them a block of **G** at a time, every iteration, exactly as `addusdens` and `newd` do it. On a 45-atom slab with 21 Å of vacuum that is **8.4 MB** against **65 GB** for one relativistic nickel dataset.

```
DEFUMAT_AUG_MAX_BYTES=off python3 run.py   # never tabulate
DEFUMAT_AUG_MAX_BYTES=2G  python3 run.py   # the default
DEFUMAT_AUG_CHUNK=8192    python3 run.py   # G vectors per block
```

`DEFUMAT_AUG_CHUNK` sizes the rebuild's working array, $(n_h, n_h, \mathrm{chunk})$, and defaults to putting it near 256 MB; like the batching dials it is a loop bound over an exact sum and is invisible in a result. The two schemes agree to **4e-10 Ry** on a total energy and exactly at $\mathbf{G} = 0$. On eight-atom ultrasoft silicon the two contractions cost about **4.5x** more from the table, and the whole run comes out level, because the table is cheaper to build than the stored array is.

Past the end of the table is NaN, not an extrapolation. The table reaches $2\sqrt{\texttt{ecutrho}}$ — Quantum ESPRESSO's own `cell_factor` — so that a strained cell, whose G vectors move while the G *set* does not, still lands on it. A strain large enough to take $|\mathbf{G}|$ past that gives NaN rather than a clamped value, because a clamped $Q^L(q)$ would be smooth, of the right order and wrong, and would show up only as a bad stress.

Precision. Every dtype comes from `config.Precision`; x64 is enabled before any array exists, and all validation is float64. float32 is a performance mode and **never one a correctness claim is made in**.

14 Accuracy summary

Against Quantum ESPRESSO on the same input:

quantity	agreement
total energy, norm-conserving LDA	$\sim$1e-9 Ry
ultrasoft, PAW	$\leq$3e-9 Ry
PBE / revPBE / PBEsol	$\leq$6e-9 Ry
metals, every smearing and tetrahedra	2.5e-8 Ry
collinear spin	1.2e-9 Ry
noncollinear magnetism	2.8e-9 Ry
spin-orbit coupling	$\leq$1.3e-8 Ry
DFT+U	$\leq$6.7e-9 Ry
band structure	0.0002 eV
forces	$\leq$2e-5 Ry/bohr
stress (13 cases)	$\leq$2.7e-7 Ry/bohr^3
relaxed geometry	1e-6 bohr
dielectric constant (NC, US, PAW)	$\leq$1.2e-4
Born effective charges (NC)	every printed digit
phonons at Γ, silicon	0.05 cm^{-1}
phonons at Γ, silicon (US, PAW)	0.02–0.03 cm^{-1}
phonons at Γ, aluminium (metal)	0.002 cm^{-1}
Raman/IR spectra vs `dynmat.x`	every printed digit

One case does not close, and is recorded rather than explained. `bi10-soc` — ten bismuth atoms with a fully-relativistic dataset — sits **1.9e-4 Ry** from QE on a total of -1477.737, which is 1.3e-7 relative. Both codes choose the same symmetry operations, the same k-point, the same grid and the same G-vectors, and both converge. It is in the test suite at that tolerance, not hidden.

Where to go next

- `README.md` — the short tour and a first calculation
- `notebooks/` — 26 worked examples on concrete systems, executed and committed, each with a `.md` export beside it
- `PERFORMANCE.md` — the running performance log, CPU and GPU
- `PLAN.md` — the architecture, the phase breakdown, the trap catalogue
- `GPU.md` — the GPU roadmap